

Word Representations

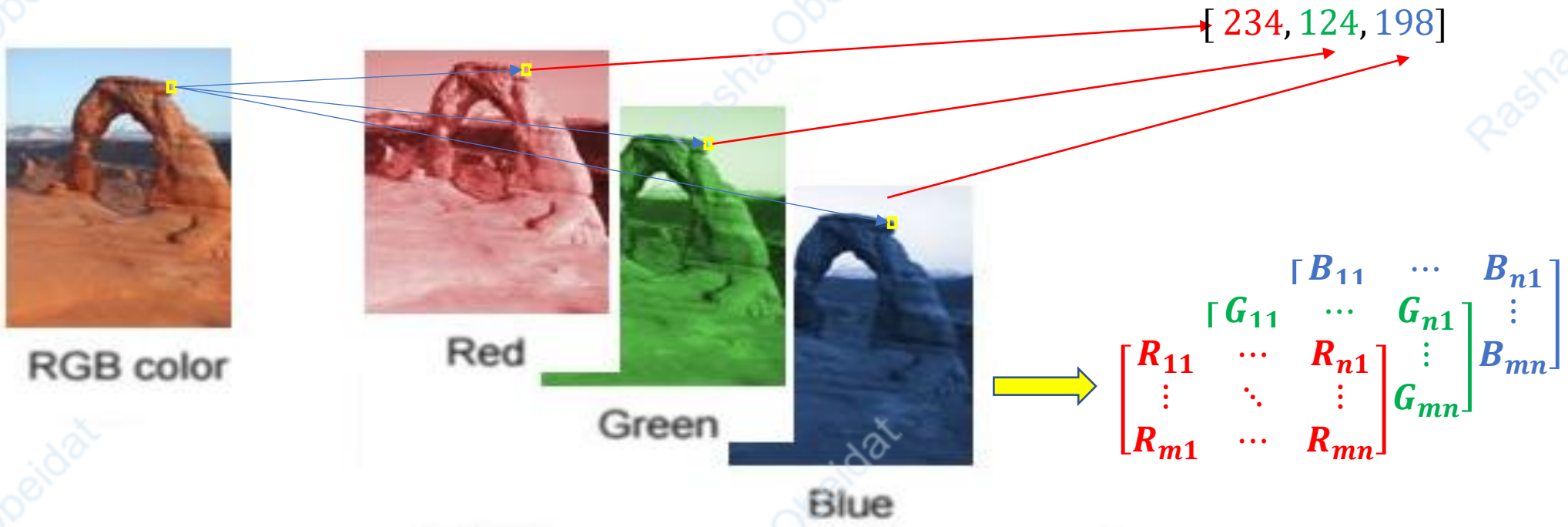
Rasha Obeidat, PhD

Natural Language Processing

Rasha Obeidat

Input Representation

- All deep learning algorithm requires that input to be converted to vector or a matrix of real values.
- For Example, Images are converted to matrices of pixel values before being fed to convolutional Neural networks.



Rasha Obeidat

Word Representation

Word Representation: techniques for encoding words as numerical vectors so that computers can process and learn from text.

- Text must be converted into ***numeric vectors*** before it can be used in ML/DL models.
  - Computers operate on numbers and vectors, not raw text.
  - Numerical representations allow models to compute similarity, detect patterns, and learn relationships between words.

Types of Word Representations

- **One-Hot Encoding**
  - Each word is represented by a vector with **one 1 and the rest 0s**
  - Indicates **whether a word exists in the vocabulary**
  - Ignores **frequency and semantic similarity between words**
- **Count-Based Representations** *(e.g., Latent Semantic Indexing (LSI))*
  - Represent words using **frequency or co-occurrence statistics**
  - Capture **limited semantic relationships**
- **Dense Representations (Word Embeddings)**
  (e.g., Word2Vec, GloVe, FastText)
  - Represent words as **dense vectors in a low-dimensional space**
  - **Similar words have similar vector representations**

One-Hot Encoding

- **One-hot encoding is the simplest technique for representing words as vectors.**

- Let V be the vocabulary of all words in the dataset.
- Each word $w \in V$ is represented by a vector $v \in \mathbb{R}^{|V|}$
- v contains all zeros except a 1 at the index of w.
- The vector length equals the vocabulary size $|V|$.
- Examples: $v_{\text{بيت}}$, $v_{\text{دار}}$, $v_{\text{مدرسة}}$

$$V = \begin{bmatrix} \text{أ} \\ \text{أب} \\ . \\ . \\ \text{بيت} \\ . \\ . \\ \text{دار} \\ . \\ . \\ \text{مدرسة} \\ . \\ . \end{bmatrix} \Rightarrow v_{\text{بيت}} = \begin{bmatrix} 0 \\ 0 \\ . \\ . \\ 1 \\ . \\ . \\ 0 \\ . \\ . \\ 0 \\ . \\ . \end{bmatrix} \quad v_{\text{دار}} = \begin{bmatrix} 0 \\ 0 \\ . \\ . \\ 0 \\ . \\ . \\ 1 \\ . \\ . \\ 0 \\ . \\ . \end{bmatrix} \quad v_{\text{مدرسة}} = \begin{bmatrix} 0 \\ 0 \\ . \\ . \\ 0 \\ . \\ . \\ 0 \\ . \\ . \\ 1 \\ . \\ . \end{bmatrix}$$

The Vocabulary

Limitations of one-hot-encoding

1. High Dimensionality

- Vocabulary size can be **hundreds of thousands or millions**
- Each word vector has size $|V|$, Leads to the **curse of dimensionality**
- **Curse of Dimensionality** refers to the phenomenon that occurs when classifying or analyzing ***high-dimensional data*** which negatively impacting the efficiency and performance of Machine Learning models.
  - As the dimensionality increases, the feature space expands exponentially, making learning more difficult.
  - This leads to a higher demand for training data, computational resources, and storage space,

2. Sparse Vectors

- Most vector values are **0,** and only **one element is 1**
- Causes inefficient storage and computation
- Makes learning difficult for many ML algorithms

Limitations of one-hot-encoding

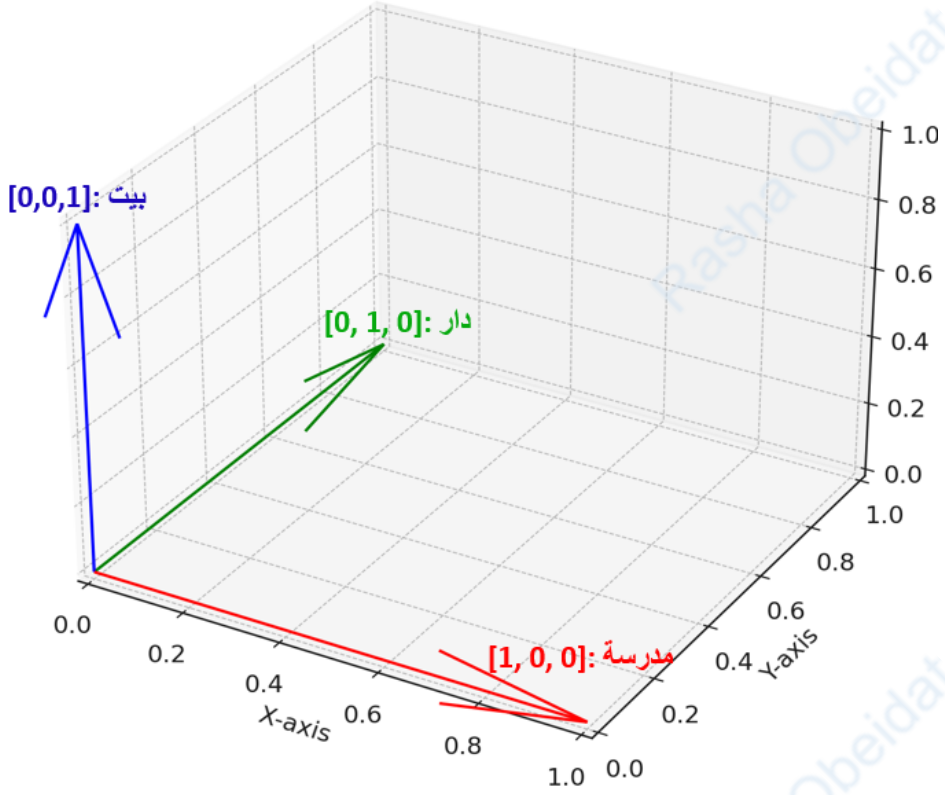
3. Doesn't provide a notion of similarity (lack semantic relationships**):** one-hot encoding treat words as completely independent points in a high dimensional *orthogonal* vector space.

Example:

$$\text{cosine_similarity}\left(v_{\text{بيت}}, v_{\text{مدرسة}}\right)$$

$$= \text{cosine_similarity}\left(v_{\text{بيت}}, v_{\text{دار}}\right)$$

$$= \text{cosine_similarity}\left(v_{\text{مدرسة}}, v_{\text{دار}}\right) = 0$$



Cosine Similarity

- **Cosine similarity** measures the **cosine of the angle** between two non-zero vectors in a multi-dimensional space.
  - It indicates how similar two vectors are.
  - It depends on the direction of the vectors, not their magnitude.

$$cosSim(v,u) = \frac{v.u}{\|v\|.\|u\|} \qquad \|v\| = \sqrt{\sum_{i=1}^{d} v_i^2}$$

  - $v.u$ is the dot product of the two vectors, $v, u \in \mathbb{R}^d$.
  - $\| v \|$ and $\| u \|$ are the **vector magnitudes (Euclidean norms)**

Cosine Similarity

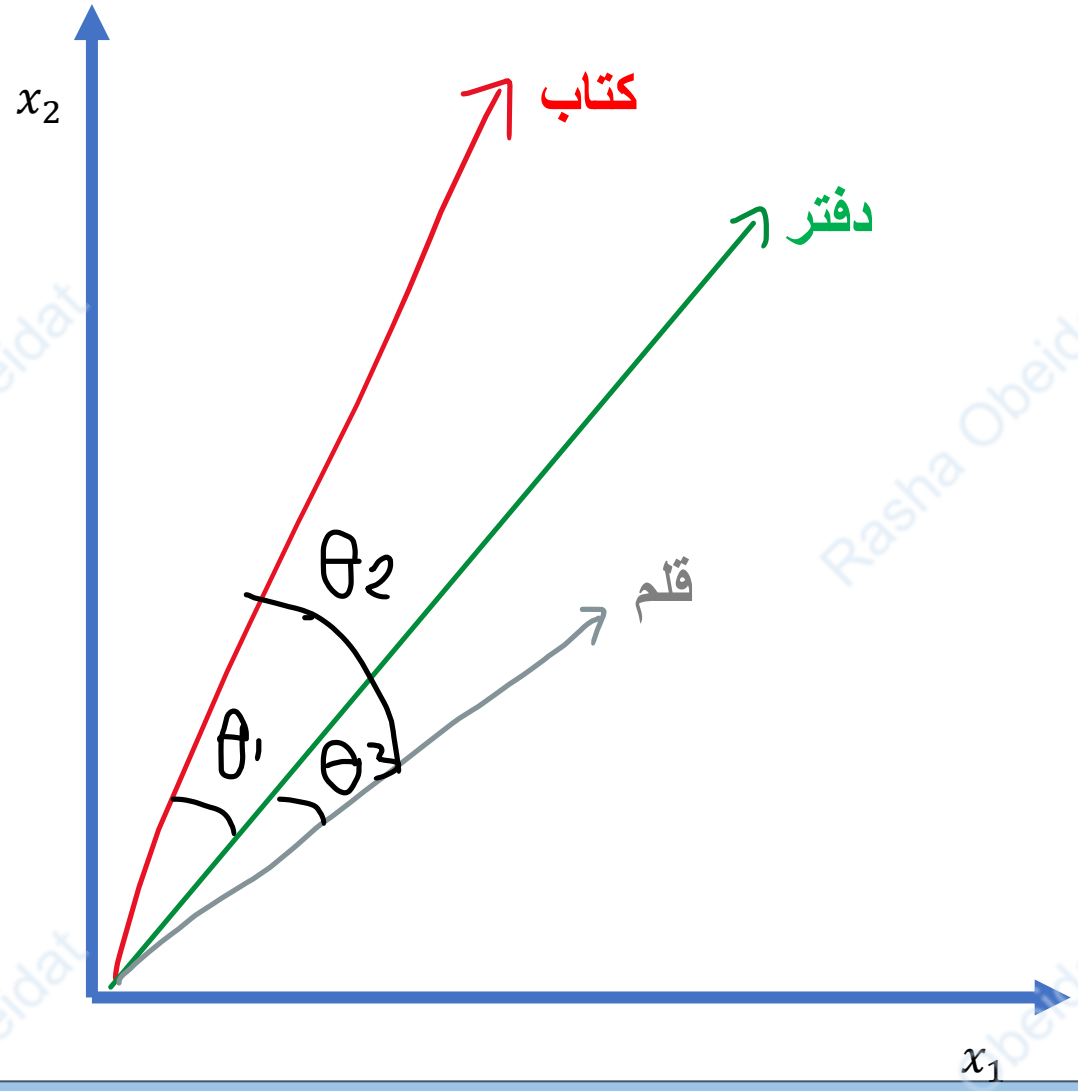
- **Properties of Cosine Similarity**
  - **Range**: The cosine similarity value is between **-1 and 1**.
    - **1** → Vectors are identical (point in the same direction).
    - **0** → Vectors are orthogonal (completely dissimilar).
    - **-1** → Vectors are opposite (point in opposite directions).
  - **Scale Invariance: ** Since cosine similarity considers only the angle, the magnitude of the vectors does not affect the result (because of dividing by $\|v\|.\|u\|$) .

Cosine Similarity

- **Cosine as a Similarity Measure**
- **Angle is small** → Cosine has a **large** value
- **Angle is large** → Cosine has a **small** value
- Example :

$$\text{cosine_sim}\left(v_{\text{كتاب}}, v_{\text{دفتر}}\right) > \text{cosine_sim}\left(v_{\text{كتاب}}, v_{\text{قلم}}\right)$$

$$\text{cosine_sim}\left(v_{\text{قلم}}, v_{\text{دفتر}}\right) > \text{cosine_sim}\left(v_{\text{كتاب}}, v_{\text{قلم}}\right)$$



Word Representation

- **Our goal** → Find low dimensional subspace (much lower than $\mathcal{R}^{1\times|V|}$) that encodes semantic relationships between words:
  - The words that have similar meaning are projected close-by in the vector space.
  - Similar words occur in similar contexts are exchangeable (**paradigmatic similarity**).

ادى { الطلاب / الطلبة / التلاميذ } جميعا تحية العلم

  - **Paradigmatic Similarity** refers to the relationship between words that can be substituted for one another in a given context while maintaining a similar meaning.
  - Transfer learning for text: allow for automatic generalization .

Task-specific Embeddings

- **A possible solution**: learn a task-specific word embeddings by

1. Add a layer called ***embedding layer*** to the deep neural network of the task at hand, by representing each word in the vocabulary by a vector in a low-dimensional space ***that randomly initialized***.

2. Embed each word in the input example by that vector and allow **back-propagation (or back-propagation through time as in RNNs)** the error to fine-tunes the values of these word vectors.

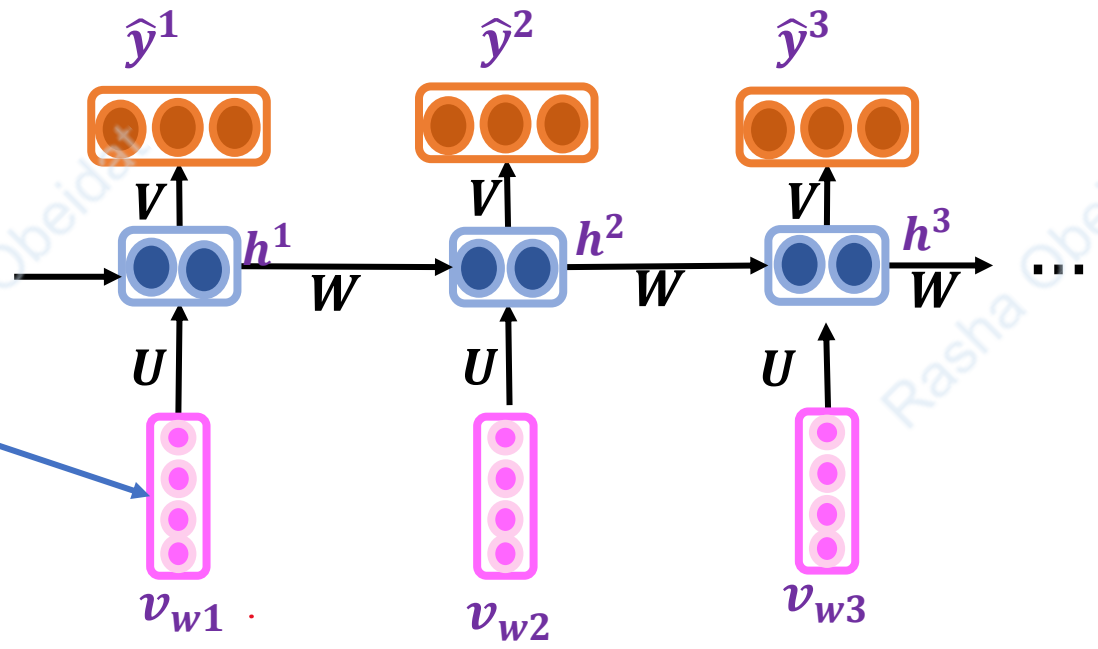
   - The error (loss) is computed from the difference between the ground truth values and the predicted results of the current task

   - The fine-tuned vectors represent the weights of the word embeddings.

Task-specific Embeddings

- E.g., in the following RNN diagram, not only U, V, and W are learned, the word vectors v_{w1}, v_{w2}.. etc. are learned as well

This vector representation is shared between all of the occurrences of the word w_3 in the training data.



- The more the word appear in the dataset, the more its corresponding emending is updated by back-propagation through time.

Task-specific embeddings

- **Shortcomings**:

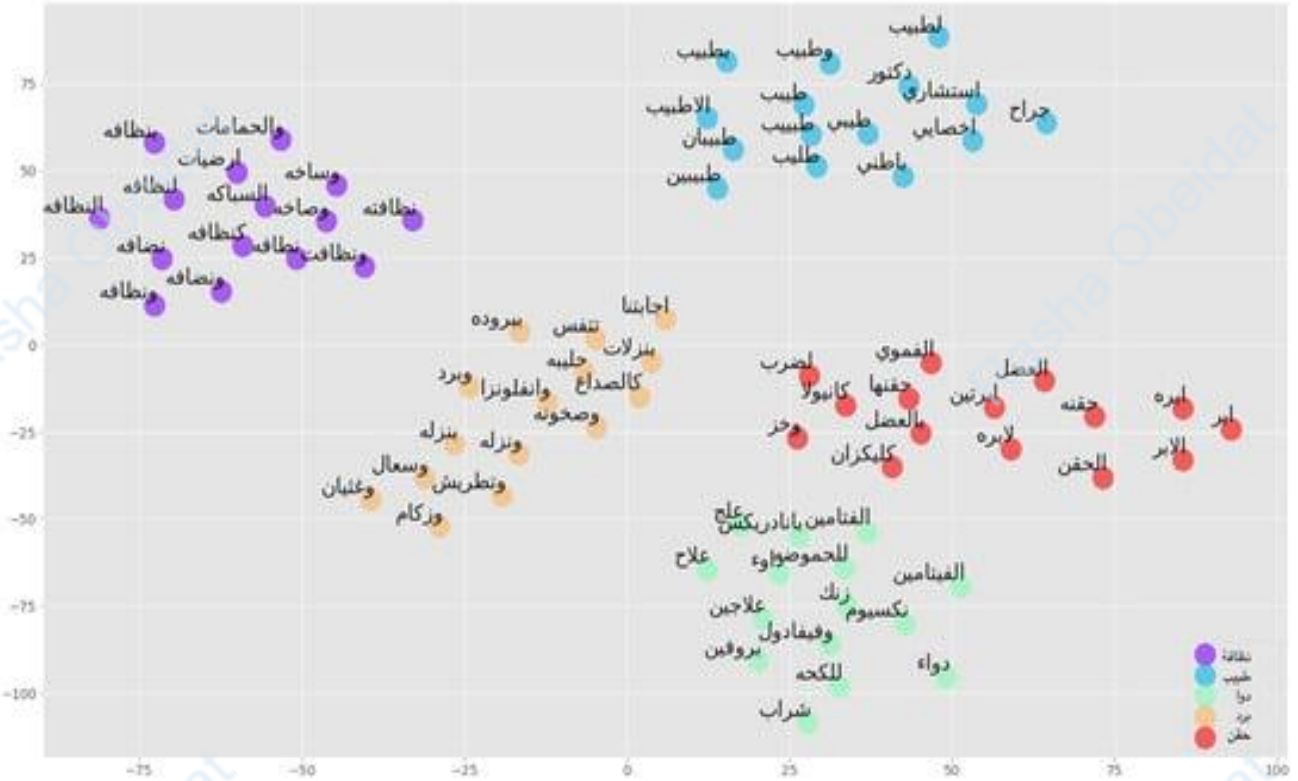
1. Task-specific embedding closely coupled to these tasks, making transferring them to be used in a different task (i.e., Transfer Learning) other than the task used to learn them *inapplicable* due to the differences in vocabulary distributions between these tasks.

2. Fail to capture contextual meaning of the word in many cases, as it is learned to serve that specific task.

3. Their quality highly depends on the available amount of data which is consider limited in most cases.

Pre-trained word embeddings

- Pretrained word vectors are learned from big amounts of unsupervised data (built just by reading huge corpus).
- Can then be used in downstream tasks and give a good performance even using smaller amounts of labeled data.
- Assumes dependence between words.
- Called an "**embedding**" because it's *embedded* into a space



https://www.mdpi.com/2076-3417/13/18/10305

Types of Word Embeddings

1. Context-*independent* word embeddings

2. Context-*dependent* word embeddings

Rasha Obeidat

Context-independent Word Embeddings

- Fixed numerical representations of words, independent of their usage in different contexts.
- Every word has a **single vector representation**, regardless of its meaning in a sentence.
- **Examples :** Word2Vec (CBOW & Skip-Gram), GloVe, and FastText
- **Limitations:**
  - Fails to capture polysemy and contextual variations.For instance, the word (سلم) in the two sentence (سلم الرجل على صديقه) and (المسلم من سلم المسلمون من لسانه ويده) have the same word embedding even they have different meanings.
  - **Biases**: model learns statistical relationships **without understanding their ethical implications** "man is to computer programmer as woman is to homemaker" “doctor – man + woman = nurse”

Context-dependent Word Embeddings

- Unlike Context-independent embeddings, these representations are **context-aware** (change based on context).
- The same word has **different embeddings** depending on how it is used in a sentence.
- **Examples:** ELMo , BERT , GPT .
- **Advantages**
  - Handles Polysemy
  - **Improves Performance:** Better understanding of semantic meaning in real-world NLP tasks→ effective in MT, QA, text generation and other task.

Rasha Obeidat

Context-dependent Word Embeddings

- **Limitations:**
  - **High Computational Cost**: require **significant memory and processing power**
  - **Lack of Interpretability**: Unlike simpler models (e.g., Word2Vec), transformer-based embeddings are hard to interpret.
  - **Domain Adaptation Challenges**: pretrained embeddings may **not generalize well** to highly specialized fields (e.g., medical or legal texts). Fine-tuning is often required, adding **extra computational cost**.
  - **Bias in Pretrained Models**: Contextual embeddings can inherit **biases from training data**, leading to **Occupational**, **gender, racial, or cultural biases** in NLP applications. Example , in "The nurse said that he/she would help." BERT is **more likely to predict "she"** due to gender stereotypes.

Word2vec

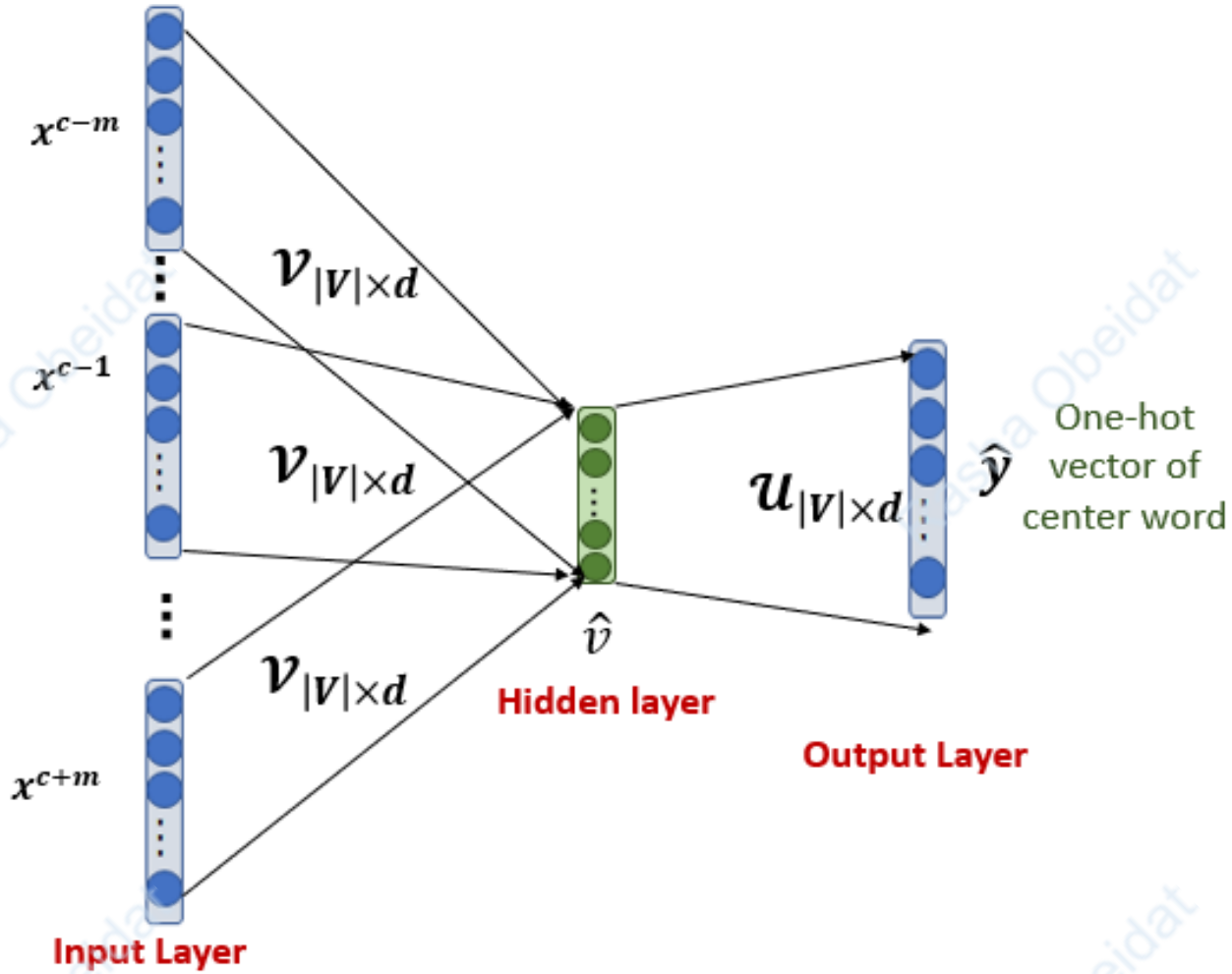
- **Word2vec** A neural-based model for **word embeddings**, introduced by Mikolov et al. (2013).
- Maps words to **dense, continuous vector representations** based on their context.
- Use iterative approaches for learning word embeddings from raw text.
- They are one of the primary reason for NLP's breakout.
- Iterative approaches.
- **Main intuition**: the words that appear in similar context should have similar vector representation:
  - E.g., $similarity\left(v_{\text{بيت}}, v_{\text{دار}}\right) > similarity\left(v_{\text{بيت}}, v_{\text{مدرسة}}\right)$
- learn the probability of a word given its context.

Word2vec

- To get word2vec word embeddings we train a shallow neural network with a single hidden layer to predict a center word from its the surrounding context or the opposite (predict the probabilities) of the context words given the center word)
- The Word2vec tool contains two models
  1. **Continuous Bag-Of-Words**: learn to predict a center word from given the surrounding context.
  2. **Skip-gram**: learn to predict the distribution (probability) of context words given the center word.

Continuous Bag-Of-Words (CBOW)

- In the **CBOW** model, the embeddings of context (or neighboring words) are combined to predict the central **word (the word in the middle)**.
- CBOW learn to predict the central word from the context



Continuous Bag-Of-Words (CBOW)

- The context word are specified by sliding a ***fixed-size window (2m)*** over the text. The window-size is a hyper-parameter.
- It assumes that the central word w^c generated based on the context words $w^{c-m}, \ldots, w^{c-1}, w^{c+1}, \ldots, w^{c+m}$.
- CBOW models conditional probability of generating the middle word, i.e., $p(w^c | w^{c-m}, \ldots, w^{c-1}, w^{c+1}, \ldots, w^{c+m})$

Preparing training data for Word2vec

- CBOW models are trained using large amounts of unlabeled text data.
  - Examples: Large corpora such as Wikipedia, news articles, books, and web text are collected.
  - The text is not manually labeled (unsupervised learning).
- Creating Training Examples

  The text is processed sentence by sentence.

  A sliding context window moves across each sentence.

  For each position in the window, the center word becomes the target word and the The surrounding words become the context words
- Handling Sentence Boundaries

  When the context window extends beyond the beginning or end of a sentence, special padding tokens (<pad>) are added.
- Each sentence generates many training examples, how many?

Generating Training Examples for Word2vec

<pad> <pad> هناك فرق بين التعليم الالكتروني والتعليم عن بعد<pad> <pad>

Central word	The Context words
هناك	فرق بين<pad> <pad>

Generating Training Examples for Word2vec

<pad> <pad>عن بعد والتعليم الالكتروني بين التعليم **فرق** هناك <pad> <pad>

Central word	The Context words
هناك	<pad> <pad>فرق بين
فرق	<pad>هناك بين التعليم

Rasha Obeidat

Generating Training Examples for Word2vec

<pad> <pad> هناك فرق بين التعليم الالكتروني والتعليم عن بعد<pad> <pad>

Central word	The Context words
هناك	فرق بين<pad> <pad>
فرق	هناك بين التعليم<pad>
بين	هناك فرق التعليم الالكتروني

Generating Training Examples for Word2vec

<pad> <pad> هناك فرق بين التعليم الالكتروني والتعليم عن بعد <pad> <pad>

Central word	The Context words
هناك	<pad> <pad> فرق بين
فرق	<pad> هناك بين التعليم
بين	هناك فرق التعليم الالكتروني
التعليم	فرق بين الالكتروني و

Generating Training Examples for Word2vec

<pad> <pad> هناك فرق بين التعليم الالكتروني والتعليم عن بعد <pad> <pad>

Central word	The Context words
هناك	<pad> <pad>فرق بين
فرق	<pad>هناك بين التعليم
بين	هناك فرق التعليم الالكتروني
التعليم	فرق بين الالكتروني و
الالكتروني	بين التعليم والتعليم

Generating Training Examples for Word2vec

<pad> <pad> هناك فرق بين التعليم الالكتروني والتعليم عن بعد <pad> <pad>

Central word	The Context words
هناك	<pad> <pad>فرق بين
فرق	<pad>هناك بين التعليم
بين	هناك فرق التعليم الالكتروني
التعليم	فرق بين الالكتروني و
الالكتروني	بين التعليم والتعليم
و	التعليم الالكتروني التعليم عن

Generating Training Examples for Word2vec

<pad> <pad> هناك فرق بين التعليم الالكتروني والتعليم عن بعد <pad> <pad>

Central word	The Context words
هناك	<pad> <pad> فرق بين
فرق	<pad> هناك بين التعليم
بين	هناك فرق التعليم الالكتروني
التعليم	فرق بين الالكتروني و
الالكتروني	بين التعليم والتعليم
و	التعليم الالكتروني التعليم عن
التعليم	الالكتروني وعن بعد

Generating Training Examples for Word2vec

<pad> <pad> هناك فرق بين التعليم الالكتروني والتعليم عن بعد <pad>

Central word	The Context words
هناك	<pad> <pad> فرق بين
فرق	<pad> هناك بين التعليم
بين	هناك فرق التعليم الالكتروني
التعليم	فرق بين الالكتروني و
الالكتروني	بين التعليم والتعليم
و	التعليم الالكتروني التعليم عن
التعليم	الالكتروني وعن بعد
عن	والتعليم عن بعد <pad>

Generating Training Examples for Word2vec

<pad> <pad> هناك فرق بين التعليم الالكتروني والتعليم عن بعد <pad> <pad>

Central word	The Context words
هناك	<pad> <pad>فرق بين
فرق	<pad>هناك بين التعليم
بين	هناك فرق التعليم الالكتروني
التعليم	فرق بين الالكتروني و
الالكتروني	بين التعليم والتعليم
و	التعليم الالكتروني التعليم عن
التعليم	الالكتروني وعن بعد
عن	والتعليم عن بعد <pad>
بعد	التعليم عن <pad> <pad>

CBOW Model

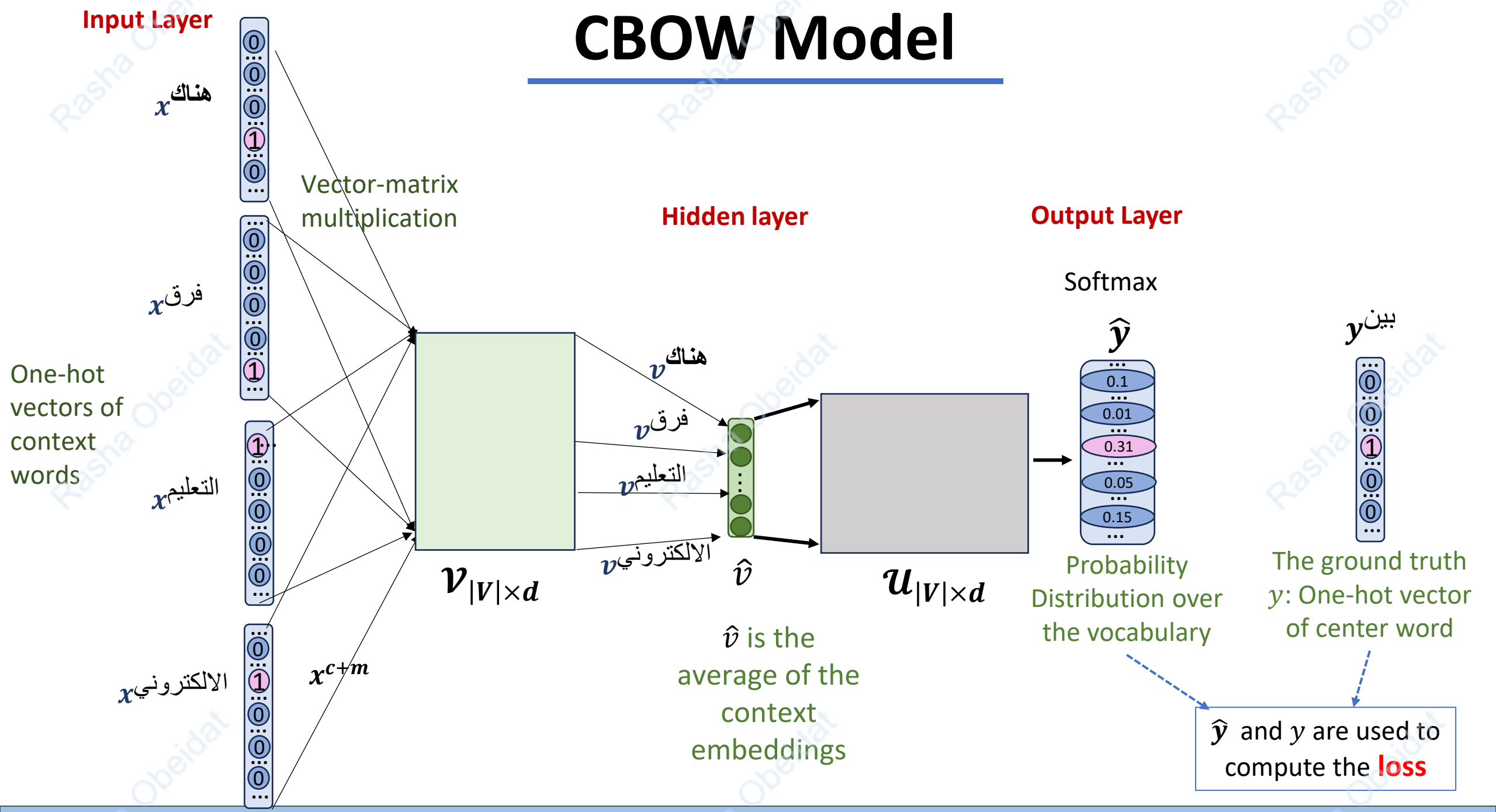
- In this model, every word $\boldsymbol{w^i}$ in the vocabulary V is represented by two d **-** dimension vectors: the **input vector** $v^i \in \mathcal{R}^{\boldsymbol{d}}$, and the **output vector** $u^i \in \mathcal{R}^{\boldsymbol{d}}$, where i is the index of the word in the vocabulary V.

- We learn the values of the embedding vectors u^i and v^i from the data for every word w^i. This means that we learn two embedding matrices $\mathcal{V} \in \mathcal{R}^{|V| \times d}$ and $\mathcal{U} \in \mathcal{R}^{|V| \times d}$ for all words in the vocabulary V.

- Vectors in $\mathcal{V}$ and $\mathcal{U}$ are used to compute the conditional probabilities $\boldsymbol{p(w^c | w^{c-m}, \ldots, w^{c-1}, w^{c+1}, \ldots, w^{c+m})}$ for all training examples obtained from the textual data.

- **The input to the model** is the one-hot vectors of the context words, i.e., $\boldsymbol{x^{c-m}, \ldots, x^{c-1}, x^{c+1}, \ldots, x^{c+m}} \in \mathcal{R}^{|V|}$, and **the output** is one-hot vector $\hat{\boldsymbol{y}}^{\boldsymbol{c}} \in \mathcal{R}^{|V|}$ or *simply* $\hat{\boldsymbol{y}}$of the central word $\boldsymbol{w^c}$.

CBOW Model


Input Layer
هناك x
فرق x
التعليم x
الالكتروني x
One-hot vectors of context words
Vector-matrix multiplication
x^{c+m}
Hidden layer
v هناك
v فرق
v التعليم
v الالكتروني
$\mathcal{V}_{|V|\times d}$
$\hat{v}$
$\hat{v}$ is the average of the context embeddings
$\mathcal{U}_{|V|\times d}$
Output Layer
Softmax
$\hat{y}$
0.1
0.01
0.31
0.05
0.15
Probability Distribution over the vocabulary
بين y
The ground truth y: One-hot vector of center word
$\hat{y}$ and y are used to compute the loss

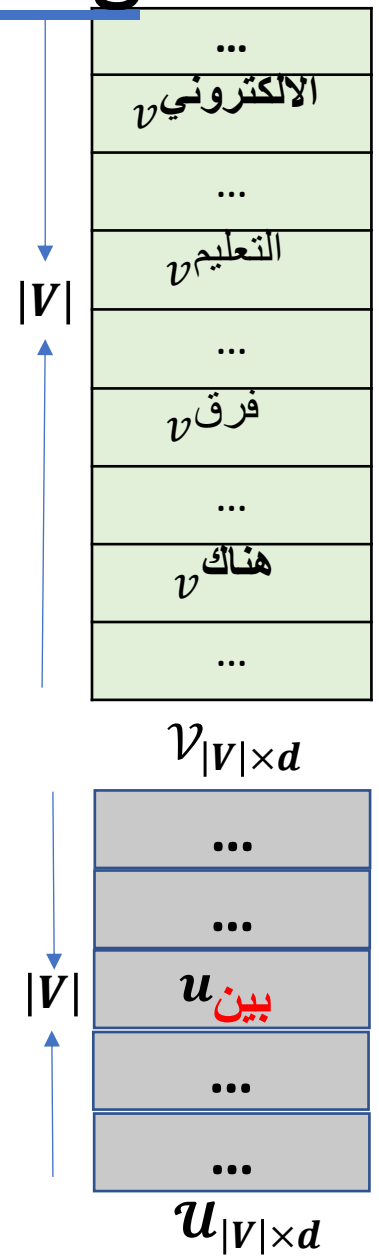
CBOW Training

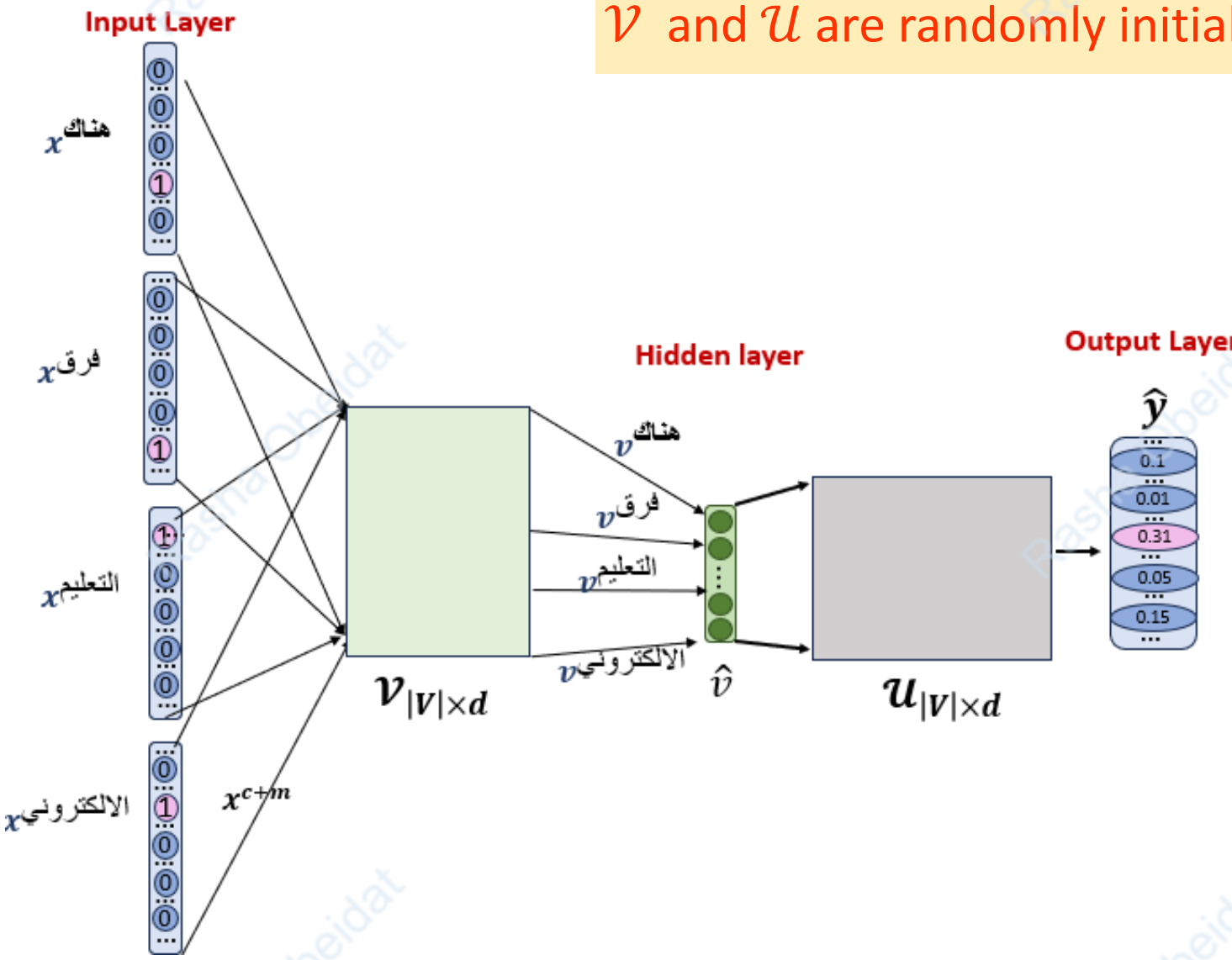
<pad> <pad> هناك فرق بين التعليم الالكتروني والتعليم عن بعد <pad> <pad>

$\mathcal{V}$ and $\mathcal{U}$ are randomly initialized

1	<pad>
...	...
53	الالكتروني
...	...
301	التعليم
...	...
1807	بعد
...	...
1923	بين
...	...
43278	عن
...	...
86542	فرق
...	...
94563	هناك
...	...
99871	و
....	...
$\|V\|$	.

The vocabulary V





CBOW Model: Forward pass

- **In the forward-pass**, we preform the following:
  - ❑Obtain the input word embedding of every word w^i in the context as the dot product of the one-hot vector of the word $\boldsymbol{o}^i$ by the matrix $\mathcal{V}$ as follows:

$$\boldsymbol{v}^{c-m} = \mathcal{V}^T \boldsymbol{x}^{c-m}, \dots, \boldsymbol{v}^{c-1} = \mathcal{V}^T \boldsymbol{x}^{c-1}, \dots, \boldsymbol{v}^{c+1} = \mathcal{V}^T \boldsymbol{x}^{c+1}, \boldsymbol{v}^{c+m} = \mathcal{V}^T \boldsymbol{x}^{c+m}$$

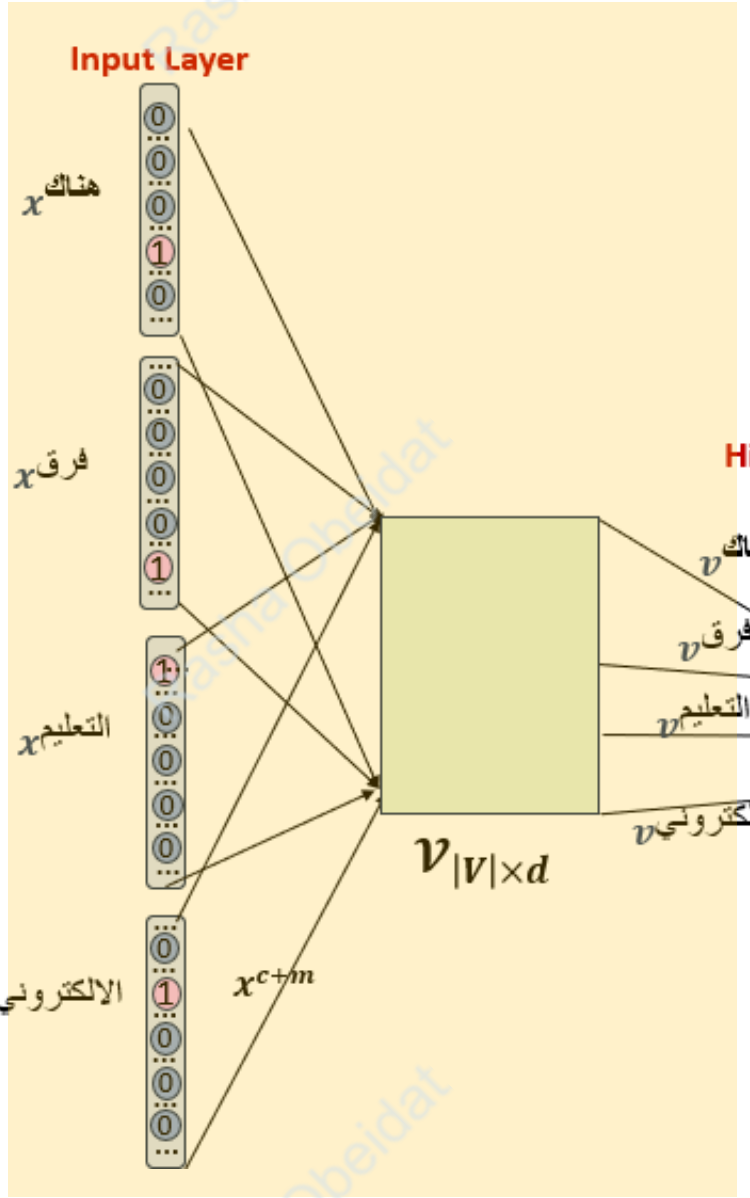
Forward pass

<pad> <pad> هناك فرق **بين** التعليم الالكتروني والتعليم عن بعد <pad> <pad>

1	<pad>
...	...
53	الالكتروني
...	...
301	التعليم
...	...
1807	بعد
...	...
1923	بين
...	...
43278	عن
...	...
86542	فرق
...	...
94563	هناك
...	...
99871	و
....	...
$\|V\|$	.

The vocabulary V



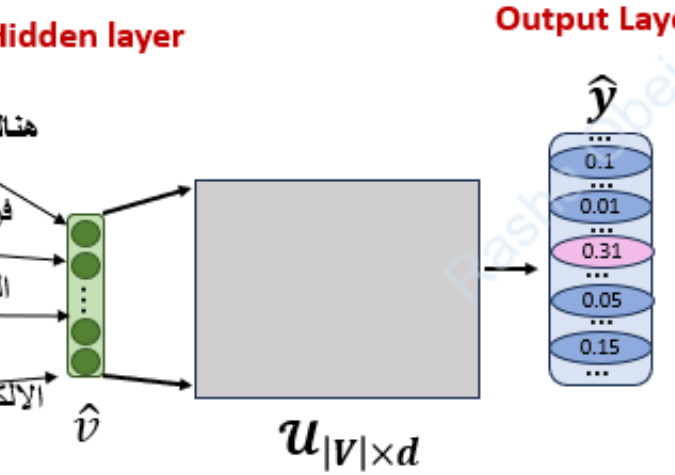


$v^{\text{هناك}} = \mathcal{V}^T x^{\text{هناك}}$

$v^{\text{فرق}} = \mathcal{V}^T x^{\text{فرق}}$

$v^{\text{التعليم}} = \mathcal{V}^T x^{\text{التعليم}}$

$v^{\text{الالكتروني}} = \mathcal{V}^T x^{\text{الالكتروني}}$



CBOW Model: Forward pass

- **In the forward-pass**, we preform the following:
  - Obtain the input word embedding of every word w^i in the context as the dot product of the one-hot vector of the word $\boldsymbol{o}^i$ by the matrix $\mathcal{V}$ as follows:

$$\boldsymbol{v}^{c-m} = \mathcal{V}^T \boldsymbol{x}^{c-m}, \dots, \boldsymbol{v}^{c-1} = \mathcal{V}^T \boldsymbol{x}^{c-1}, \dots, \boldsymbol{v}^{c+1} = \mathcal{V}^T \boldsymbol{x}^{c+1}, \boldsymbol{v}^{c+m} = \mathcal{V}^T \boldsymbol{x}^{c+m}$$

  - Compute the context vector $\hat{\boldsymbol{v}} \in \mathcal{R}^d$ as the average of the vectors of context words:

$$\hat{\boldsymbol{v}} = \frac{\mathbf{1}}{\mathbf{2m}}\left(\boldsymbol{v}^{c-m} +, \dots, + \boldsymbol{v}^{c-1} + \boldsymbol{v}^{c+1} +, \dots, + \boldsymbol{v}^{c+m}\right) = \frac{\mathbf{1}}{\mathbf{2m}} \sum_{\boldsymbol{i}=-\boldsymbol{m}}^{\boldsymbol{m}} \boldsymbol{v}^{c+i}$$

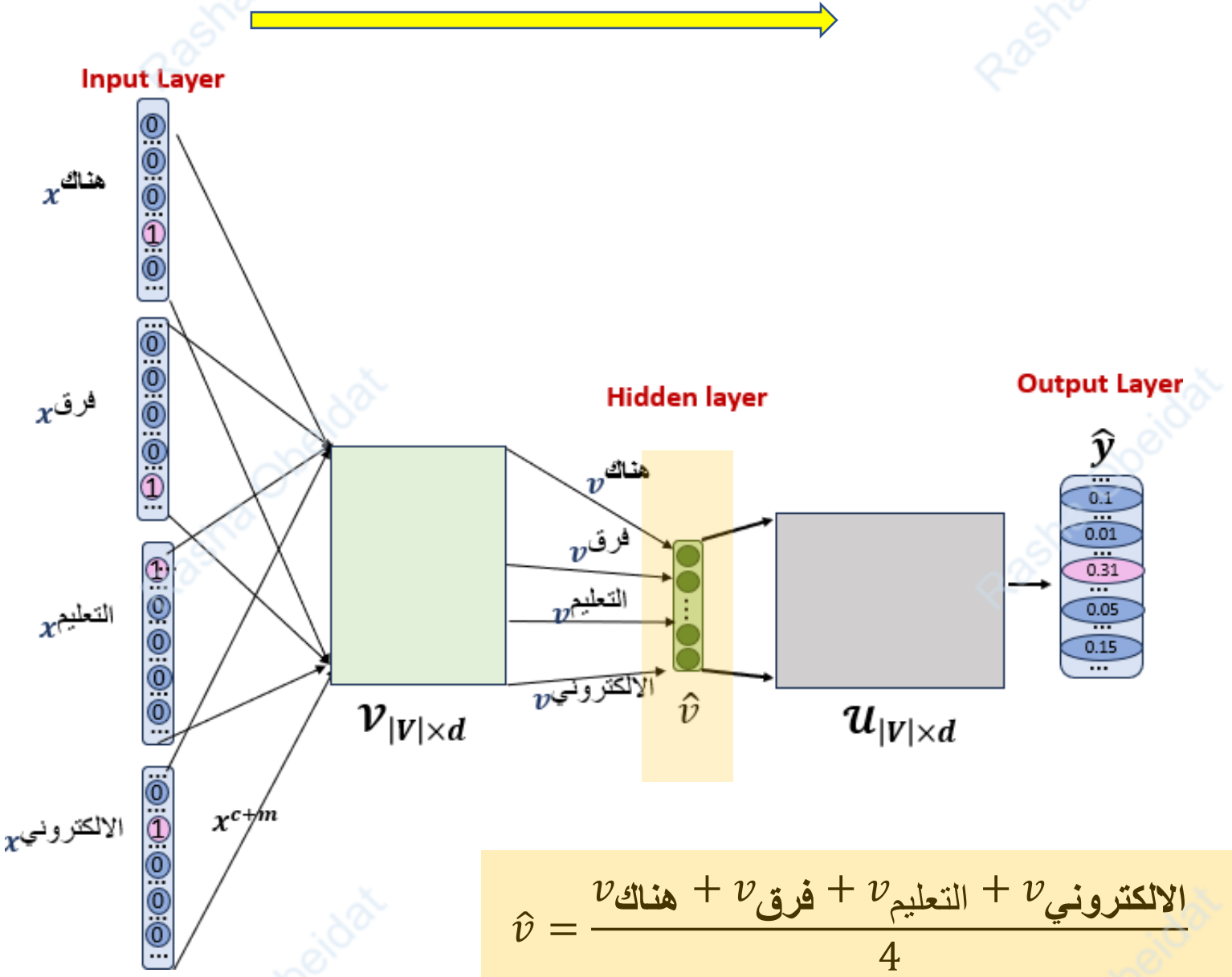
Forward pass

<pad> <pad> هناك فرق بين التعليم الالكتروني والتعليم عن بعد <pad> <pad>

1	<pad>
...	...
53	الالكتروني
...	...
301	التعليم
...	...
1807	بعد
...	...
1923	بين
...	...
43278	عن
...	...
86542	فرق
...	...
94563	هناك
...	...
99871	و
....	...
$\|V\|$	.

The vocabulary V





$$\hat{v} = \frac{v_{\text{هناك}} + v_{\text{فرق}} + v_{\text{التعليم}} + v_{\text{الالكتروني}}}{4}$$

CBOW Model: Forward pass

- **In the forward-pass**, we preform the following:
  - ❑Obtain the input word embedding of every word w^i in the context as the dot product of the one-hot vector of the word $\boldsymbol{o}^i$ by the matrix $\mathcal{V}$ as follows:

$$\boldsymbol{v}^{c-m} = \mathcal{V}^T \boldsymbol{x}^{c-m}, \dots, \boldsymbol{v}^{c-1} = \mathcal{V}^T \boldsymbol{x}^{c-1}, \dots, \boldsymbol{v}^{c+1} = \mathcal{V}^T \boldsymbol{x}^{c+1}, \boldsymbol{v}^{c+m} = \mathcal{V}^T \boldsymbol{x}^{c+m}$$

  - ❑ Compute the context vector $\hat{\boldsymbol{v}} \in \mathcal{R}^d$ as the average of the vectors of context words:

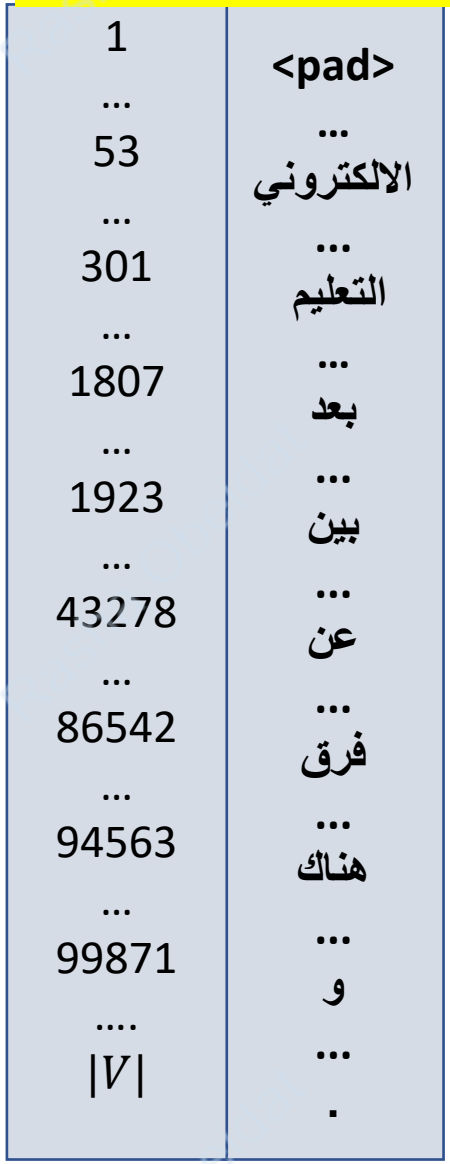
$$\hat{\boldsymbol{v}} = \frac{\mathbf{1}}{\boldsymbol{2m}}\left(\boldsymbol{v}^{c-m} +, \dots, +\boldsymbol{v}^{c-1} + \boldsymbol{v}^{c+1} +, \dots, +\boldsymbol{v}^{c+m}\right) = \frac{\mathbf{1}}{\boldsymbol{2m}} \sum_{\boldsymbol{i=-m}}^{\boldsymbol{m}} \boldsymbol{v}^{c+i}$$

  - ❑Compute the score vector $\boldsymbol{z} \in \mathcal{R}^{|V|}$ which has the unnormalized score of very word in the vocabulary V against the context vector $\hat{\boldsymbol{v}}$ as dot product of the output embedding matrix $\mathcal{U}$ and the context vector $\hat{\boldsymbol{v}}$ (i.e., $\boldsymbol{z} = \mathcal{U}\hat{\boldsymbol{v}}$).
  - ❑Turn the scores into probabilities vector $\hat{\boldsymbol{y}}$ over the whole vocabulary by using the $softmax$.

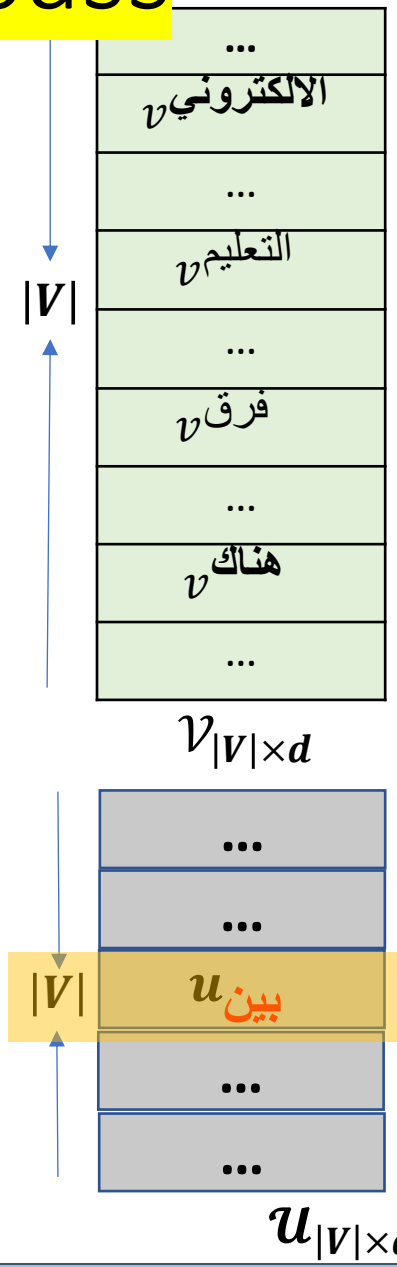
$$\hat{y} = softmax(z)\,, \hat{y}^{\boldsymbol{i}} = \frac{\exp\left((\mathrm{u}^{\boldsymbol{i}})^{\boldsymbol{T}}\, \hat{v}\right)}{\sum_{k\in\{1,\dots|V|\}} \exp\left((\mathrm{u}^{\boldsymbol{k}})^{\boldsymbol{T}}\, \hat{v}\right)}, \quad \boldsymbol{i} \in \{1, \dots |V|\}$$

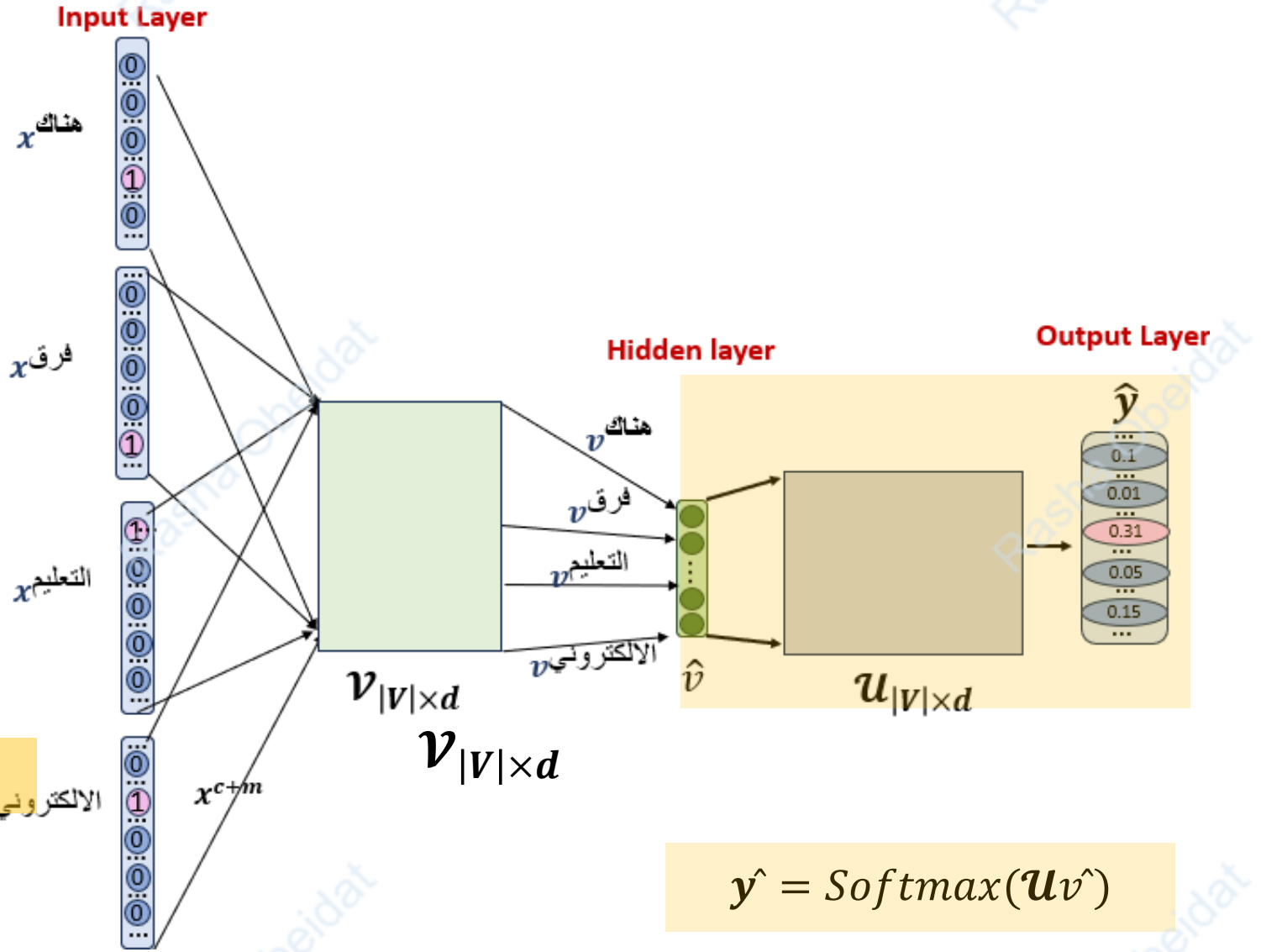
Forward pass

<pad> <pad> هناك فرق بين التعليم الالكتروني والتعليم عن بعد <pad> <pad>



The vocabulary V





$$\hat{y} = Softmax(\mathcal{U}\hat{v})$$

CBOW Training – Backward Pass

- **In the Backward–pass**: the goal is to update the embedding matrices $\mathcal{V}$ and $\mathcal{U}$ using the gradient of the loss.

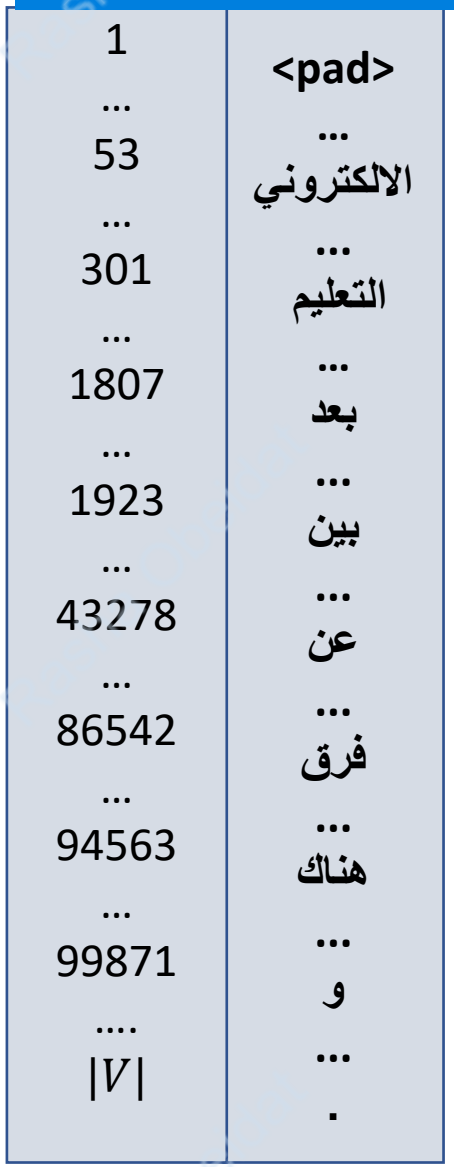
- Then ***back-propagated*** to adjusted the parts of input and the output word embedding in $\mathcal{V}$ and $\mathcal{U}$ that have been involved in the forward computations. $\mathcal{V}$ and $\mathcal{U}$ are optimized using batch or mini-batch(recommended) or online-gradient decent gradient decent using the following update rules:

$$\mathcal{V} \leftarrow \mathcal{V} - \eta \frac{\partial L}{\partial \mathcal{V}} \ ,\mathcal{U} \leftarrow \mathcal{U} - \eta \frac{\partial L}{\partial \mathcal{U}}.$$

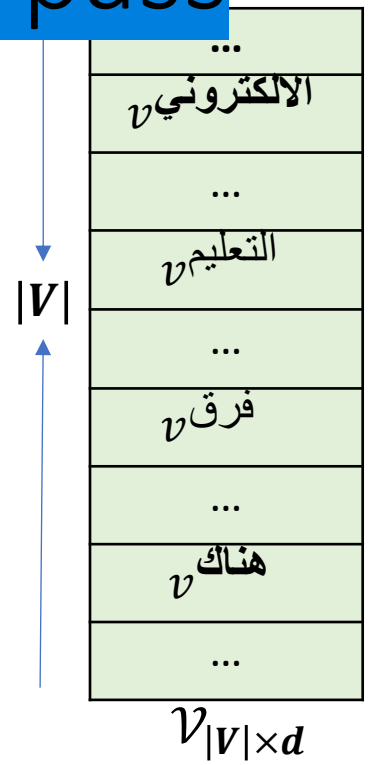
- The error (loss) can be accumulated in different ways:
  - **Batch Gradient Descent**: Update the parameters after computing the loss over the entire training dataset.
  - **Mini-Batch Gradient Descent** *(recommended):* Update the parameters after computing the loss over a small batch of training examples.
  - **Stochastic Gradient Descent (SGD):** Update the parameters after each training example.

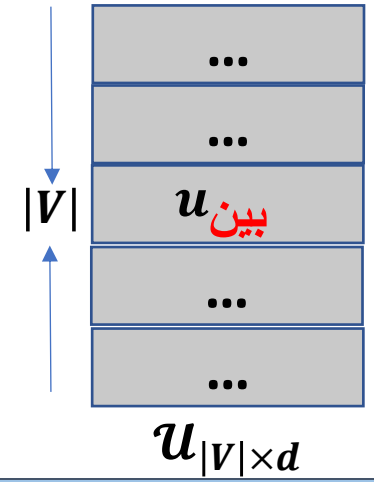
backward pass

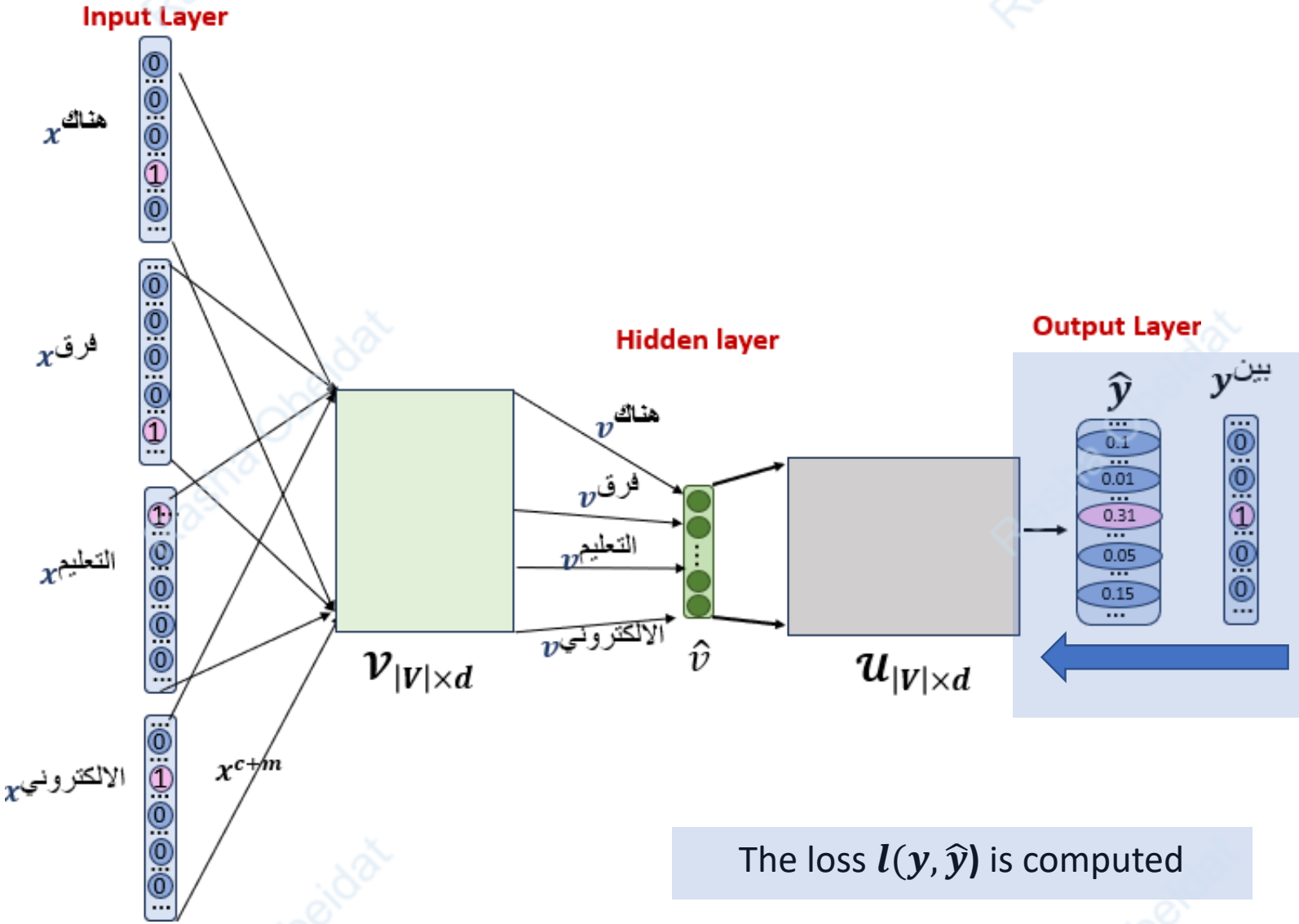
<pad> <pad> هناك فرق بين التعليم الالكتروني والتعليم عن بعد <pad> <pad>



The vocabulary V







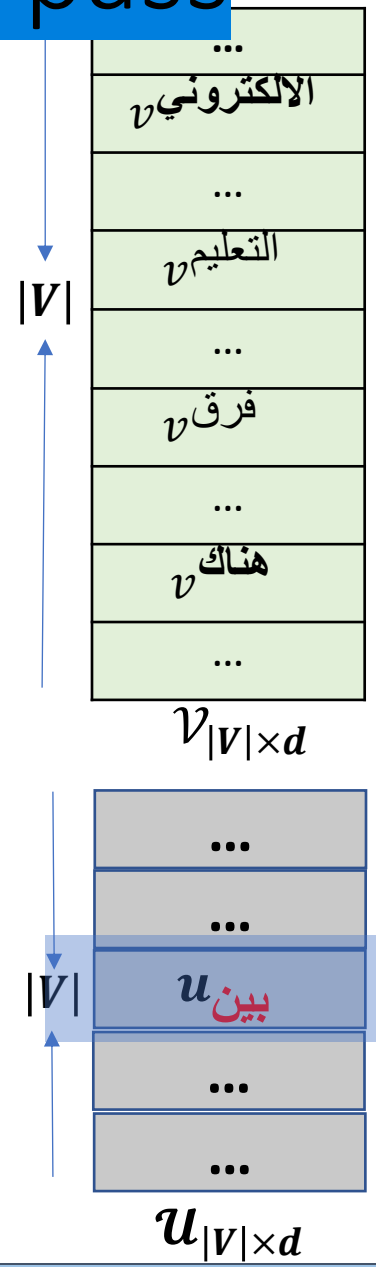
The loss $l(y, \hat{y})$ is computed

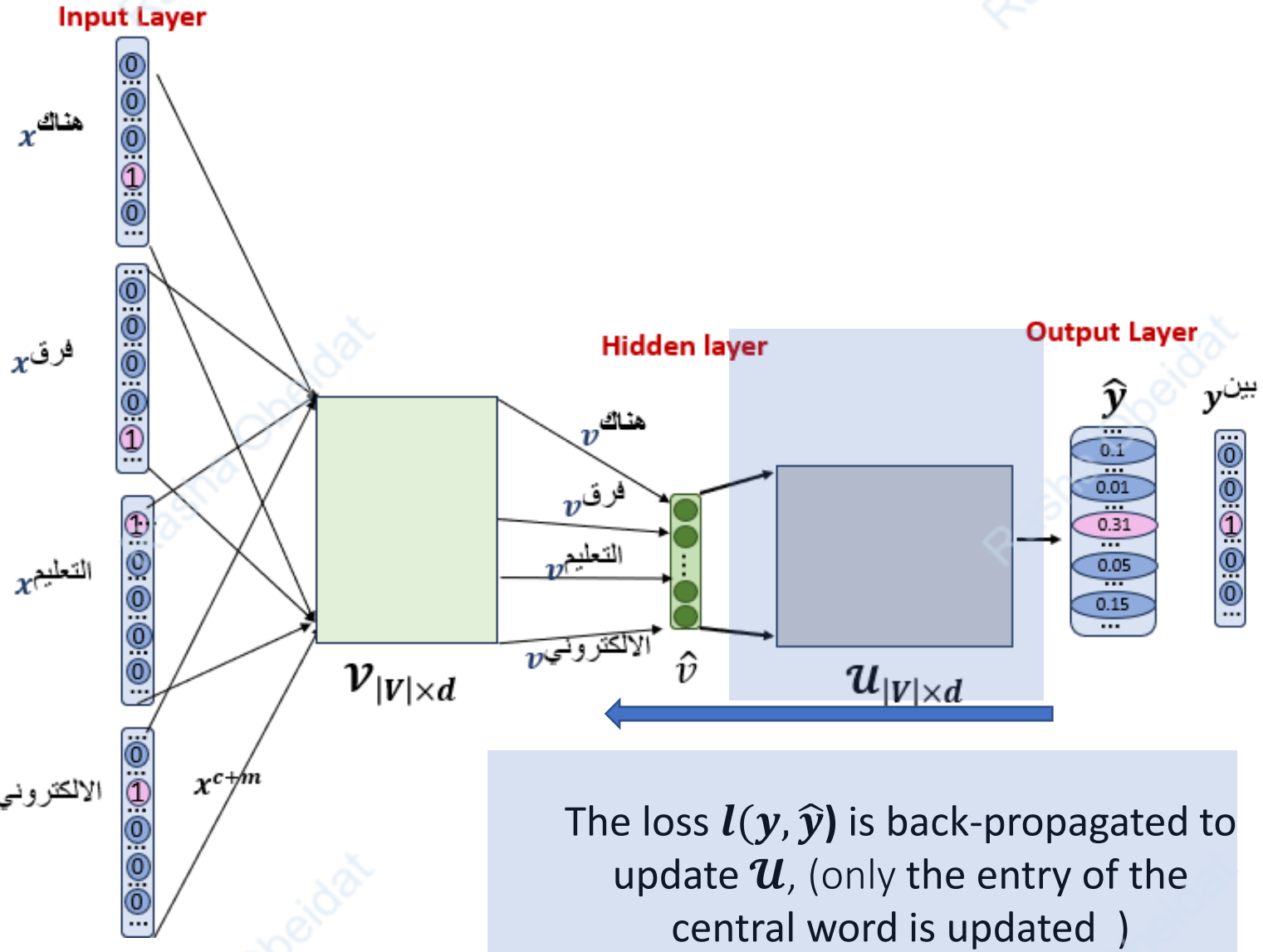
backward pass

<pad> <pad> هناك فرق بين التعليم الالكتروني والتعليم عن بعد <pad> <pad>

1	<pad>
...	...
53	الالكتروني
...	...
301	التعليم
...	...
1807	بعد
...	...
1923	بين
...	...
43278	عن
...	...
86542	فرق
...	...
94563	هناك
...	...
99871	و
....	...
$\|V\|$	.

The vocabulary V





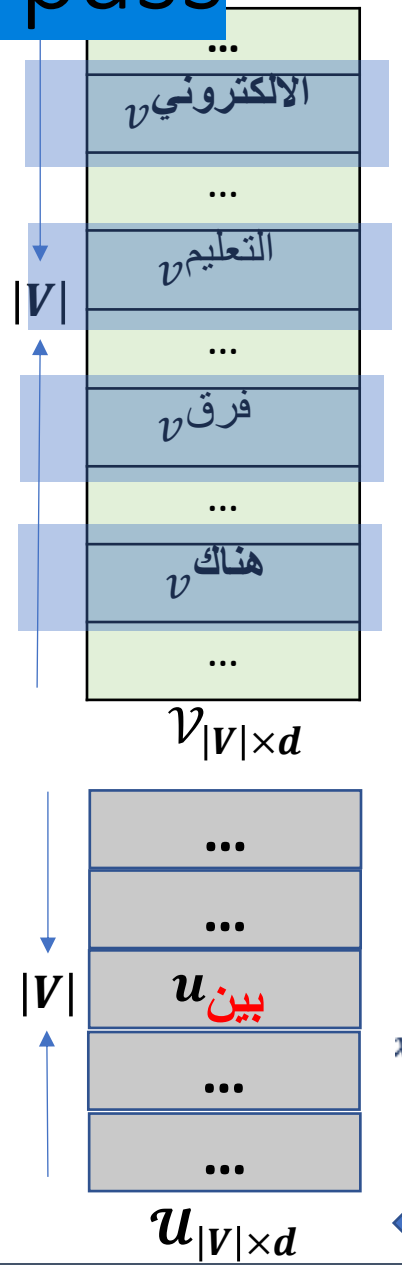
The loss $l(y, \hat{y})$ is back-propagated to update $\mathcal{U}$, (only the entry of the central word is updated)

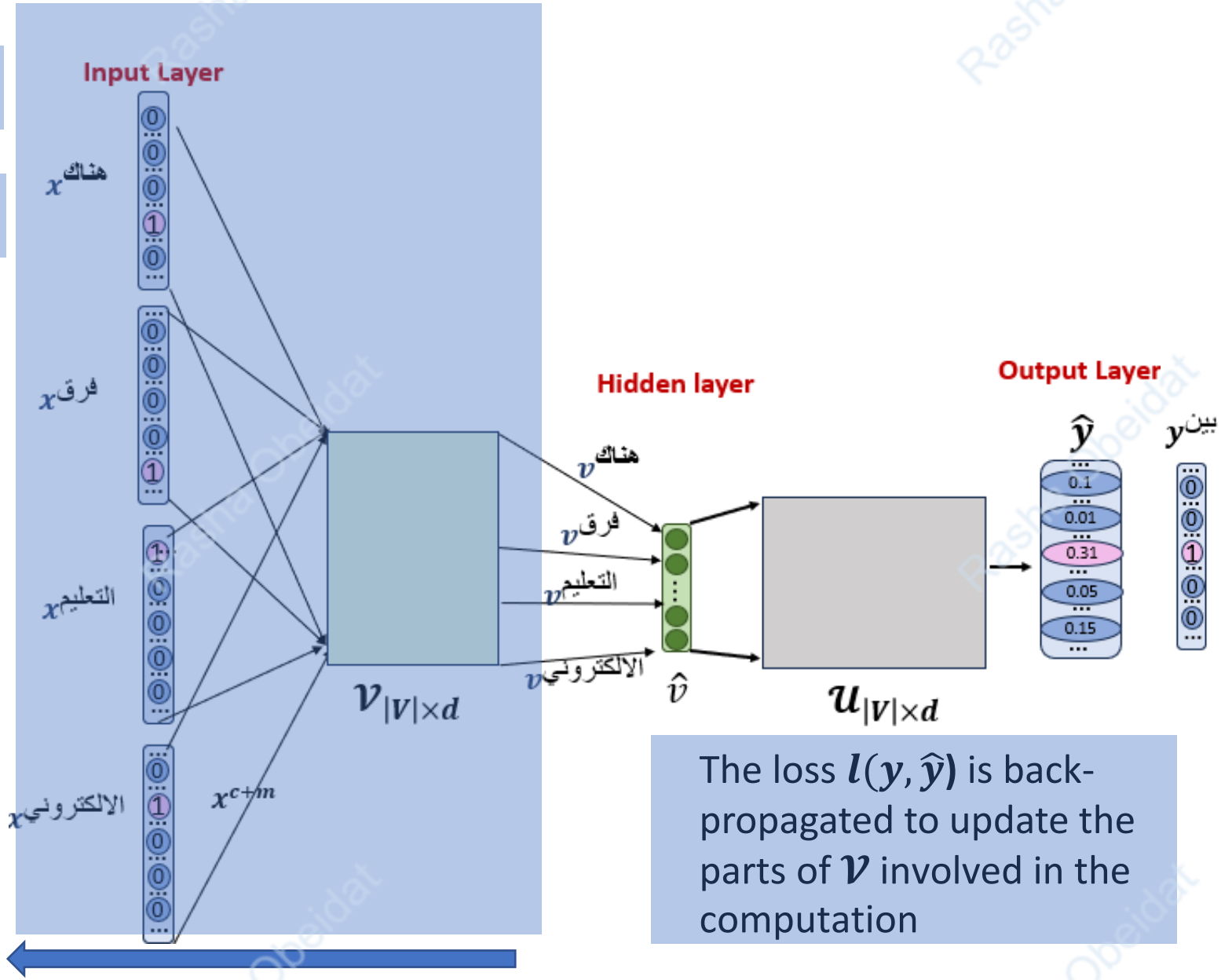
backward pass

<pad> <pad> هناك فرق بين التعليم الالكتروني والتعليم عن بعد <pad> <pad>

1	<pad>
...	...
53	الالكتروني
...	...
301	التعليم
...	...
1807	بعد
...	...
1923	بين
...	...
43278	عن
...	...
86542	فرق
...	...
94563	هناك
...	...
99871	و
....	...
$\|V\|$	.

The vocabulary V





The loss $l(y, \hat{y})$ is back-propagated to update the parts of $\mathcal{V}$ involved in the computation

CBOW Loss

- **The goal** is to learn $\mathcal{V}$ and $\mathcal{U}$ such that the more word is that are semantically related the context, the higher the probability it should get, in other words, the predicted distribution vector $\hat{\boldsymbol{y}}$ over the whole vocabulary matches $\boldsymbol{y}$ as much as possible.

- Formally, learn $\mathcal{V}$ and $\mathcal{U}$ to minimize some loss function (e.g., multi-class cross-entropy) of $\hat{\boldsymbol{y}}$ **and** $\boldsymbol{y}$ over all training examples. The loss over one training example is computed as follows:

$$l(y, \hat{y}) = -y^T \, log \hat{y}$$

- where:
  - y= **one-hot target vector** (size $|\ V\ |$)
  - $\hat{y}$ =**predicted probability vector** from softmax
  - $\log(\hat{y})$ =elementwise log
- Or equivalently

$$l(y, \hat{y}) = -\sum_{i \in \{1,\ldots|V|\}} y_i \log \hat{y}_i$$

CBOW Loss

- Minimizing the cross entropy is equivalent to maximizing the likelihood over the training data(n is the number of examples):

$$\prod_{c=1}^{n} p(w^c | w^{c-m}, \dots, w^{c-1}, w^{c+1}, \dots, w^{c+m})$$

- And The probability of the target word is computed using **softmax**:

$$\hat{y} = P(w^c \mid w^{c-m}, \dots, w^{c-1}, w^{c+1}, \dots, w^{c+m}) = \frac{\exp\left((u^c)^T \hat{v}\right)}{\sum_{k \in \{1,\dots|V|\}} \exp\left((u^k)^T \hat{v}\right)}$$

- So

$$l(y, \hat{y}) = -\log p(w^c | w^{c-m}, \dots, w^{c-1}, w^{c+1}, \dots, w^{c+m})$$

$$= -\log p(u^c \mid \hat{v}) = -\log \frac{\exp\left((u^c)^T \hat{v}\right)}{\sum_{k \in \{1,\dots|V|\}} \exp\left((u^k)^T \hat{v}\right)}$$

$$= -(u^c)^T \hat{v} + \log\left(\sum_{k \in \{1,\dots|V|\}} \exp\left((u^k)^T \hat{v}\right)\right)$$

Gradients

- Then ***back-propagated*** to adjusted the parts of input and the output word embedding in $\mathcal{V}$ and $\mathcal{U}$ that have been involved in the forward computations. $\mathcal{V}$ and $\mathcal{U}$ are optimized using batch or mini-batch(recommended) or online-gradient decent gradient decent using the following update rules:

$$\mathcal{V} \leftarrow \mathcal{V} - \eta \frac{\partial L}{\partial \mathcal{V}} \ , \mathcal{U} \leftarrow \mathcal{U} - \eta \frac{\partial L}{\partial \mathcal{U}}.$$

- **Gradients**:

$$\frac{\partial L}{\partial v^{c+i}} = \frac{1}{2m}(\hat{y} - y)u^c, \qquad v^{c+i} \leftarrow v^{c+i} - \eta \frac{\partial L}{\partial v^{c+i}}$$

$$\frac{\partial L}{\partial u^c} = \frac{1}{2m}(\hat{y} - y)\hat{v}, \qquad u^c \leftarrow u^c - \eta \frac{\partial L}{\partial u^c}$$

- **Important observation**: All context words receive the same gradient vector.

$$\frac{\partial L}{\partial v^{c-2}} = \frac{\partial L}{\partial v^{c-1}} = \frac{\partial L}{\partial v^{c+1}} = \frac{\partial L}{\partial v^{c+2}}$$

CBOW Training

- After training is performed , the embedding of the context words $\mathcal{V}$ are used as word embeddings.
- Understanding $\mathcal{V}$ and $\mathcal{U}$ in CBOW
  - $\mathcal{V}$ is used to embed the context words.
  - $\mathcal{U}$ is used to generate the probability distribution over words.
- The final word embeddings are taken from $\mathcal{V}$ because they best represent words in a meaningful vector space. **Why?**
  - The matrix $\mathcal{V}$ captures the **semantic relationships** between words based on their context.
  - The **context words were directly embedded** using $\mathcal{V}$, meaning this matrix holds meaningful word representations.
  - The output matrix $\mathcal{U}$ is used for classification (to predict the target word), and its rows are optimized for prediction rather than capturing general word semantics.

Skip-gram

- Skip-gram differs from CBOW model that, the embeddings of **central word** is used to predict the **context words.**
- Similar to CBOW, The context word are specified by sliding a fixed-size window over the text.
- Skip-gram assumes that the context words $\boldsymbol{w^{c-m}, \ldots, w^{c-1}, w^{c+1}, w^{c+m}}$, have been generated based on the central word $\boldsymbol{w^c}$
- Skip-gram models conditional probability of generating the context words given the central word, i.e., $\boldsymbol{p(w^{c-m}, \ldots, w^{c-1}, w^{c+1}, w^{c+m} | w^c)}$

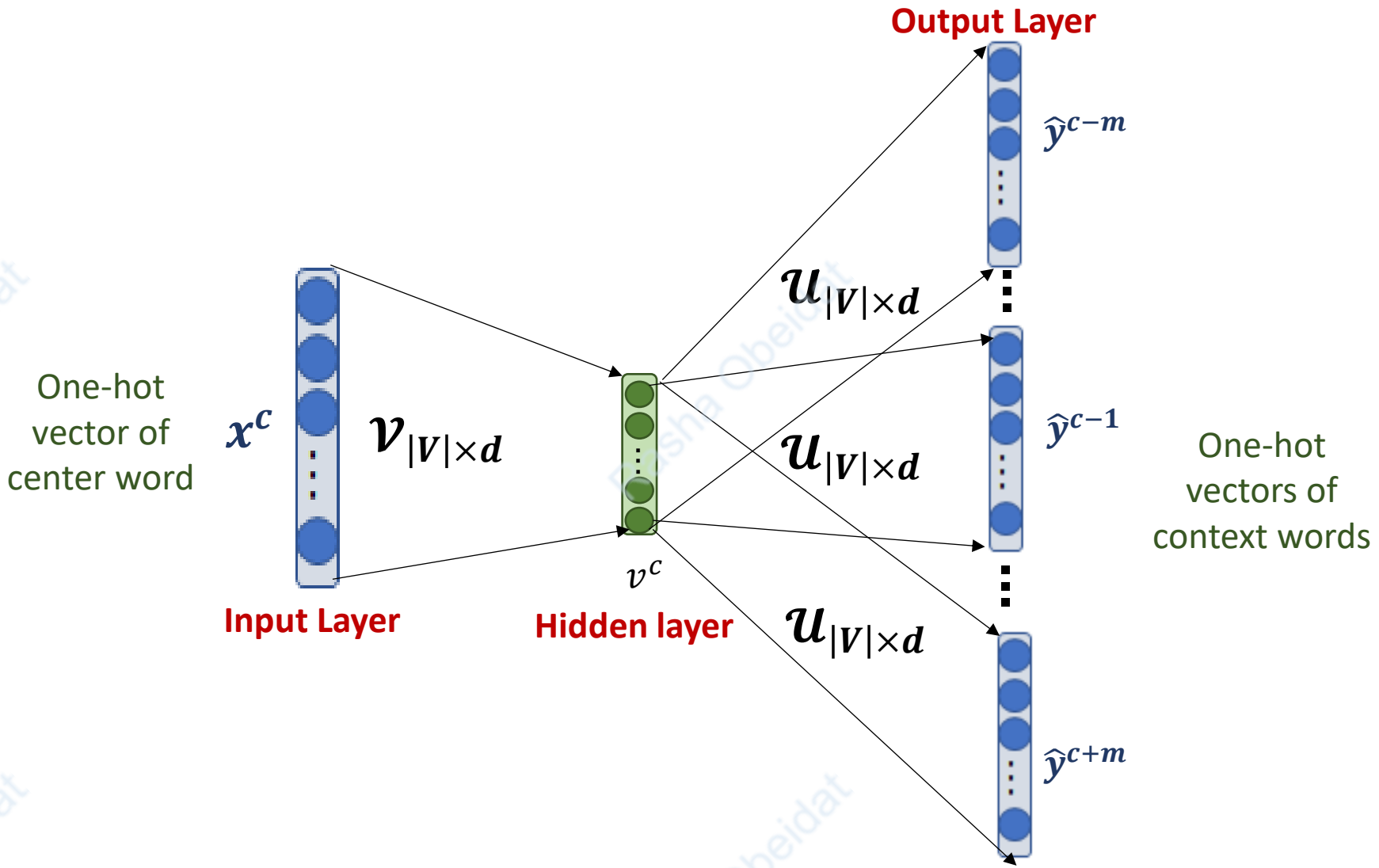
Skip-gram Model

- As in CBOW model, every word $\boldsymbol{w^i}$ in the vocabulary V is represented by two d-dimension vectors: the **input vector** $v^i \in \mathcal{R}^{\boldsymbol{d}}$, and the **output vector** $u^i \in \mathcal{R}^{\boldsymbol{d \times 1}}$, where i is the index of the word in the vocabulary V.
- $\mathcal{V}, \mathcal{U} \in R^{|V| \times d}$ are defined in the same way as in CBOW $\mathcal{V}$ and $\mathcal{U}$ are used to compute the conditional probabilities $\boldsymbol{p(w^{c-m}, \dots, w^{c-1}, w^{c+1}, \dots, w^{c+m} | w^c)}$
- By assuming that the context words are independent, we can break $\boldsymbol{p(w^{c-m}, \dots, w^{c-1}, w^{c+1}, \dots, w^{c+m} | w^c)}$ as:

$$\boldsymbol{p\left(w^{c-m}, \dots, w^{c-1}, w^{c+1}, \dots, w^{c+m} | w^c\right) = \prod_{i=-m}^{m} p(w^{c+i} | w^c)}$$

- **The input to the model** is one-hot vector $\boldsymbol{x^c} \in R^{|V|}$ of the central word $\boldsymbol{w^c}$, and the output are the probabilities of observing the context words i.e., $\boldsymbol{\hat{y}^{c-m}, \dots, \hat{y}^{c-1}, \hat{y}^{c+1}, \dots, \hat{y}^{c+m}} \in R^{|V|}$.

Skip-gram Model



Skip-gram Model

- **In the forward-pass,** we preform the following:

❑**Step 1: Obtain the target word embedding**

Obtain the input word embedding of the word w^c in the context as the dot product of the one-hot vector of the word $\boldsymbol{x^c}$ by the matrix $\mathcal{V}$:

$$\boldsymbol{v^c} = \mathcal{V}^T \boldsymbol{x^c}$$

❑**Step 2: Compute scores for all vocabulary words**

Compute the score vector $\boldsymbol{z} \in \boldsymbol{R}^{|V|}$ which has the unnormalized score of very word in the vocabulary V against the vector $\boldsymbol{v^c}$ as dot product of the output embedding matrix $\mathcal{U}$ and the context vector$\boldsymbol{v^c}$:

$$\boldsymbol{z} = \mathcal{U}\boldsymbol{v^c}$$

Each score measures how likely a word is to appear in the context of w^c

Skip-gram Model

- **In the forward-pass,** we preform the following:
  - ❑ **Step 3: Apply Softmax**

    Turn the scores into probabilities vector $\hat{y}$ over the whole vocabulary by using the $softmax$.

$$\hat{y} = softmax(z)$$

$$\hat{y}^{i} = \boldsymbol{p}(\boldsymbol{w}^{i}|\boldsymbol{w}^{c}) = \frac{\exp\left((\mathrm{u}^{i})^{T}\ \boldsymbol{v}^{c}\right)}{\sum_{k\in\{1,\dots|V|\}}\exp\left((\mathrm{u}^{k})^{T}\boldsymbol{v}^{c}\right)} \quad ,\boldsymbol{i} \in \{1,\dots|V|\}$$

Skip-gram Training

- **The goal** is to learn $\mathcal{V}$ and $\mathcal{U}$ such the predicted probability vector $\hat{\boldsymbol{y}}$ matches the one hot vectors of the context words $\boldsymbol{y^{c-m}}, \ldots, \boldsymbol{y^{c-1}}, \boldsymbol{y^{c+1}}, \ldots, \boldsymbol{y^{c+m}}$
- Each context word $\boldsymbol{x^{c-m}}, \ldots, \boldsymbol{x^{c-1}}, \boldsymbol{x^{c+1}}, \ldots, \boldsymbol{x^{c+m}}$ is treated independently. i.e., **Skip-gram decomposes the problem into multiple independent subproblems**, one per **context word**.
- Formally, learn $\mathcal{V}$ and $\mathcal{U}$ to minimize some loss function (e.g., multi-class cross-entropy) of $\hat{\boldsymbol{y}}$ **and** $\boldsymbol{y}$ over all training examples.
- **Training Objective**: for a window of size m, the loss for one center word w^c

$$l^c = -\sum_{\substack{-m \le i \le m \\ i \neq 0}} \log \boldsymbol{p}(\boldsymbol{w^{c+i}} | \boldsymbol{w^c})$$

Each $\boldsymbol{p}(\boldsymbol{w^{c+i}} | \boldsymbol{w^c})$ comes from the same probability distribution $\hat{y}$, but the model only considers one context word at a time when computing the loss corresponding to that word and then summing them up.

Skip-gram Training

- **Skip-gram Loss (All Training Data):** Let the corpus contain T words. The total loss is:

$$l = -\frac{1}{T}\sum_{c=1}^{T}\sum_{i=-m,\, i\neq 0}^{m} \log \boldsymbol{p}(\boldsymbol{w^{c+i}}|\boldsymbol{w^{c}})$$

$$= -\frac{1}{T}\sum_{c=1}^{T}\sum_{i=-m,i\neq 0}^{m} log\frac{exp\left(\left(u^{c+i}\right)^{T}v^{c}\right)}{\sum_{k\in\{1,\dots|V|\}} exp\left((u^{k})^{T}v^{c}\right)}$$

$$= \frac{1}{T}\sum_{c=1}^{T}\sum_{i=-m,i\neq 0}^{m} (u^{c+i})^{T}v^{c} + log(\sum_{k\in\{1,\dots|V|\}} exp\left((u^{k})^{T}v^{c}\right))$$

- **In the Backward–pass**: The **error (loss)** can be collected for the
  - 1) whole training data (batch gradient descent) , or
  - 2) for mini-batches (mini-batch gradient descent) or
  - 3) for one examples (Stochastic gradient descent)

Skip-gram Training

- Then ***back-propagated*** to adjust the parts of input and the output word embedding in $\mathcal{V}$ and $\mathcal{U}$ that have been involved in the forward computations. $\mathcal{V}$ and $\mathcal{U}$ are optimized using batch or mini-batch(recommended) or online gradient decent using the following update rules:

$$\mathcal{V} \leftarrow \mathcal{V} - \eta \frac{\partial L}{\partial \mathcal{V}} \ ,\mathcal{U} \leftarrow \mathcal{U} - \eta \frac{\partial L}{\partial \mathcal{U}}.$$

- Gradients:

$$\frac{\partial L}{\partial v^c} = \sum_{i=-m, i\neq 0}^{m} \left(\hat{y} - y^{c+i}\right) u^{c+i}, \qquad v^c \leftarrow v^c - \eta \frac{\partial L}{\partial v^c}$$

$$\frac{\partial L}{\partial u^{c+i}} = \left(\hat{y} - y^{c+i}\right) v^c, \qquad u^{c+i} \leftarrow u^{c+i} - \eta \frac{\partial L}{\partial u^{c+i}}$$

Skip-gram: Example

- Suppose the vocabulary has 8 words with the following order :
- ["بعد", "بين", "التعليم", "عن", "فرق", "و", "الالكتروني", "هناك"]
- Given the target word "بين", assume the correct context words are:
- "هناك", "فرق", "التعليم", "الالكتروني"

Word	One-Hot Vector
بعد	[1, 0, 0, 0, 0, 0, 0, 0]
بين	[0, 1, 0, 0, 0, 0, 0, 0]
التعليم	[0, 0, 1, 0, 0, 0, 0, 0]
عن	[0, 0, 0, 1, 0, 0, 0, 0]
فرق	[0, 0, 0, 0, 1, 0, 0, 0]
و	[0, 0, 0, 0, 0, 1, 0, 0]
الالكتروني	[0, 0, 0, 0, 0, 0, 1, 0]
هناك	[0, 0, 0, 0, 0, 0, 0, 1]

Skip-gram: Example

- The softmax output $\hat{y}$for the target word " "بين might initially look like:
- $\hat{y}$
  $= [0.03, , 0.12, , 0.22, , 0.05, \ 0.18 \ 0.10 \ 0.15 \ 0.15]$
- This follows the vocabulary order:
- ["بعد", *"بين", "التعليم", "عن", "فرق", "و", "الالكتروني",*
  "هناك"]
- then we compute:
- $L = -\log p\left(\text{هناك} \mid \text{بين}\right) - \log p\left(\text{فرق} \mid \text{بين}\right)$
  $-\log p\left(\text{التعليم} \mid \text{بين}\right) - \log p\left(\text{الالكتروني} \mid \text{بين}\right)$
- and then sum them.

Word	(p(token| بين)
بعد	0.03
بين	0.12
التعليم	0.22
عن	0.05
فرق	0.18
و	0.10
الالكتروني	0.15
هناك	0.15

Skip-gram: Example

Context Word	Index in Vocabulary	Predicted Probability	Log Probability	Negative Log Probability
هناك	7	0.15	log(0.15)=-1.8971	1.8971
فرق	4	0.18	log(0.18)=-1.7148	1.7148
التعليم	2	0.22	log(0.22)=-1.5141	1.5141
الالكتروني	6	0.15	log(0.15)=-1.8971	1.8971

$$l = -\log(0.15) - \log(0.18) - \log(0.22) - \log(0.15)$$
$$l \approx 1.8971 + 1.7148 + 1.5141 + 1.8971$$
$$l \approx 7.0231$$

Skip-gram: Final Word Representations

- After training, $\mathcal{V}$ is used Instead of $\mathcal{U}$ as word embeddings because it captures better general-purpose semantic meanings of words, making it the preferred embedding for real-world applications.
- Why $\mathcal{V}$ is used as the final word representations?
  - $\mathcal{V}$ captures better general-purpose word meanings. The central word (target word) is retrieved from $\mathcal{V}$ and used to predict multiple context words.
  - The model adjusts $\mathcal{V}$ so that words appearing in similar contexts get similar representations.
  - Empirical studies show that $\mathcal{V}$ embeddings work better for tasks like word similarity and analogy tasks.
  - $\mathcal{U}$ is trained to predict context words, but it does not directly represent the semantic properties of a word itself.

Negative Sampling

- **Problem with Softmax:**
  - Computing the loss requires summing over the **entire vocabulary** | V |
  - Time complexity per training step:$\mathcal{O}(|\ V\ |)$
  - Very expensive when | V |is large!
- **Solution** :Negative Sampling is a way to approximate it!
- **Key Idea of Negative Sampling:** Instead of considering all words in V:
  - Keep **1 positive example** (a context word)
  - Sample k **negative words** (not a context words)
- Reformulate the task as ***binary classification***:
  - Positive pair (w^c, w^o) →label = 1
  - Negative pairs $\left(w^c, w^k\right)$ →label = 0

Negative Sampling

- **Objective Function**

$$\log \sigma\left(u_o^T v_c\right) + \sum_{k=1}^{K} \log \sigma\left(-u_k^T v_c\right)$$

  - σ :sigmoid function
  - u_o :output embedding of the **true context word**
  - u_k :embeddings of **negative samples**

- **Why it Works**
  - Reduces complexity from $\mathcal{O}(|V|) \rightarrow \mathcal{O}(K)$,where $K \ll |V|$
  - Much faster and works well in practice

- For more info, refer to Mikolov, Tomas, et al. "Distributed representations of words and phrases and their compositionality." *Advances in neural information processing systems*. 2013.

GloVe: The Main Idea

- **GloVe (Global Vectors for Word Representation)** is a word embedding model that learns vector representations of words based on statistical co-occurrence information.
- **Count-based method** that rely on *matrix factorization* (similarity but LSA) effectively leverage global statistical information; they are primarily used to capture word similarities and do poorly on tasks such as word analogy.
- **Word2Vec** learn word embeddings by making predictions in local context windows. These models demonstrate the capacity to capture complex linguistic patterns beyond word similarity but fail to make use of the global co-occurrence statistics.

GloVe: The Main Idea

- Unlike Word2Vec, which relies on local context windows, GloVe leverages a **co-occurrence matrix** to capture global relationships between words.
  - GloVe considers the overall co-occurrence statistics of words across an entire corpus, rather than just looking at nearby words (local context)
  - It shows state-of-the-art performance on the word analogy task and outperforms other current methods on several word similarity tasks.
- **Glove** uses global statistics to predict the probability of word j appearing in the context of word i with a least squares objective.

GloVe: The Main Idea

- **Key principles of GloVe:**
  - Words with similar meanings appear in similar contexts (**distributional hypothesis**).
  - The meaning of a word is derived from its co-occurrence with other words.
  - The model encodes semantic relationships into a vector space where arithmetic operations reflect word analogies (e.g., "king - man + woman ≈ queen").
- **Example :** "شتاء" frequently co-occurs with "برد" but rarely with "حر" (heat), while "صيف" co-occurs with"حر" (heat), then GloVe learns that "شتاء" is more closely related to "برد" than "حر".

GloVe : Defining Context Window

- A word co-occurs with another word if they appear **within a fixed-sized window** in a corpus.
- Example (window size = 2):
- **Sentence:** "The cat sat on the mat."

Word	Context Words (±2 window)
The	cat, sat
cat	The, sat, on
sat	The, cat, on, the
on	cat, sat, the, mat
the	sat, on, mat
mat	on, the

GloVe : Building Co-occurrence Matrix

- Each row and column represent words.
- The cells contain the number of times two words co-occur in the defined context window.
- The values indicate how often words appear together within the context window.

	the	cat	sat	on	mat
the	0	1	2	1	1
cat	1	0	1	1	0
sat	2	1	0	1	0
on	1	1	1	0	1
mat	1	0	0	1	0

Co-occurrence Matrix (window size = 2) after reading "**The cat sat on the mat**"

GloVe : Building Co-occurrence Matrix

- Populating this matrix needs a single pass through the **entire corpus** to gather the necessary statistics.
- Although this process can be computationally expensive, especially for large datasets, it is essential to understand that it performed only **one-time.**

	cat	for	is	its	looking	mat	on	sat	the
cat	0	0	1	0	1	0	1	1	2
for	0	0	1	1	1	1	0	0	0
is	1	1	0	0	1	0	0	0	1
its	0	1	0	0	1	1	0	0	0
looking	1	1	1	1	0	0	0	0	0
mat	0	1	0	1	0	0	1	0	1
on	1	0	0	0	0	1	0	1	1
sat	1	0	0	0	0	0	1	0	2
the	2	0	1	0	0	1	1	2	0

The occurrence matrix after reading

- **"The cat sat on the mat"**
- **"The cat is looing for its mat"**

GloVe : Weighting Function

- Some words, like stop words (e.g., "the," "is"), appear very frequently and dominate the co-occurrence matrix.
- GloVe applies a **weighting function** to balance frequent vs. rare words and improve learning.
- **Effect of the weighting function:**
  - **Rare words** (low co-occurrence count) retain their full influence.
  - **Frequent words** (high co-occurrence count) are down-weighted, preventing them from dominating the training process.
  - Helps in reducing the effect of stop words like "the," "is," etc.

GloVe : Weighting Function

- The weighting function is defined as:

- $$f(x_{ij}) = \begin{cases} \left(\frac{X_{ij}}{X_{max}}\right)^{\alpha} & if \quad x_{ij} < max \\ 1 & x_{ij} \geq max \end{cases}$$

- X_{ij} is the co-occurrence count of words i and j.
- X_{max} is a threshold value (e.g., 100), beyond which word co-occurrences are weighted as **1**.
- α is a smoothing parameter (typically 0.75).

GloVe : Embedding Matrix

- Initialize Word Vector matrices $V \in R^{|V| \times d}$ and $U \in R^{|V| \times d}$
- Each word i is assigned a **randomly initialized embedding** v_i.
- Each word j (context word) is also assigned a separate **context vector** u_j.
- As in word2vec , these embeddings are typically low-dimensional (e.g., 50, 100, 300 dimensions).

Training Glove

- In GloVe, the word vectors v_i and u_j are **learned** through an optimization process based on the **co-occurrence matrix** and the **least squares loss function**. Here's how they are obtained:

$$\sum_{i=1}^{|V|} \sum_{j=1}^{2m} f\left(x_{ij}\right) \left(v_i^T . u_j + b_i + b_j + \log \mathrm{X_{ij}}\right)^2$$

- **Optimization Process**
  - The loss function is minimized using **Stochastic Gradient Descent** or other optimization techniques.
  - The word vectors are **iteratively updated** until the loss function converges.
  - After training, the final word embeddings **capture semantic relationships** between words.
- You can download GloVe word embeddings from https://nlp.stanford.edu/projects/glove/

Word embeddings Evaluation

- **Intrinsic Evaluation:**
  - Measures the quality of embeddings based on linguistic properties.
  - Evaluates how well embeddings capture semantic similarity, word analogy, or clustering.
  - Fast to compute performance
- **Extrinsic Evaluation**
  - Measures how well embeddings improve downstream NLP tasks.
  - Evaluates their practical impact in real-world applications.
  - Can be slow to compute performance

Rasha Obeidat

Intrinsic Evaluation

- **Intrinsic evaluation** of word embeddings measures their **linguistic quality** by assessing how well they capture **semantic similarity, analogy relationships, and word clustering**.
- This type of evaluation is performed **independently of downstream tasks** and focuses on word-level relationships
- **Common Intrinsic evaluation Tasks**
    1. Word Similarity Tasks
    2. Word Analogy Tasks
    3. Clustering & Visualization

Rasha Obeidat

Intrinsic Evaluation : Word Analogy

- A **word analogy** expresses a relationship of the form: **"a is to b as c is to ?",** compactly we are given an incomplete analogy of the form

a:b::c:?

- Example: **"man"** is to **"king"** as **"woman"** is to **"?" → queen**
- Given words a, b, and c, we compute $\mathbf{v_b} - \mathbf{v_a} + \mathbf{v_c}$ Then find the word whose vector is **closest** (e.g., via cosine similarity) to the result.
- Formally , it is the task of identifying the word vector which ***maximizes*** the cosine similarity

$$d = aregmax_i \frac{(\mathbf{v_b} - \mathbf{v_a} + \mathbf{v_c})^T \mathbf{v_i}}{\|v_b - v_a + v_c\|. ||\mathbf{v_i}||}$$

Intrinsic Evaluation : Word Analogy

Word Analogy Datasets: Google Analogy dataset, MSR analogy dataset, etc.

Example Analogy Tasks:

- **Semantic:**
  - France : Paris :: Germany : ? → Berlin
  - Einstein : scientist :: Picasso : ? → artist
- **Syntactic:**
  - walk : walking :: run : running
  - good : better :: bad : worse

Rasha Obeidat

Word Analogy Examples

Guessing the **capital of a state** is and example on word analogy demonstrated ***semantic*** testing using word vectors.

City 1 : State containing City 1 : : City 2 : State containing City 2

Input	Result Produced
Chicago : Illinois : : Houston	Texas
Chicago : Illinois : : Philadelphia	Pennsylvania
Chicago : Illinois : : Phoenix	Arizona
Chicago : Illinois : : Dallas	Texas
Chicago : Illinois : : Jacksonville	Florida
Chicago : Illinois : : Indianapolis	Indiana
Chicago : Illinois : : Austin	Texas
Chicago : Illinois : : Detroit	Michigan
Chicago : Illinois : : Memphis	Tennessee
Chicago : Illinois : : Boston	Massachusetts

The contents of this slides are taken from :
https://web.stanford.edu/class/cs224n/readings/cs224n-2019-notes02-wordvecs2.pdf

Word analogy Examples

- Here are **syntactic** word vector analogies that test the notion of **superlative adjectives**

Input	Result Produced
bad : worst : : big	biggest
bad : worst : : bright	brightest
bad : worst : : cold	coldest
bad : worst : : cool	coolest
bad : worst : : dark	darkest
bad : worst : : easy	easiest
bad : worst : : fast	fastest
bad : worst : : good	best
bad : worst : : great	greatest

The contents of this slides are taken from :
https://web.stanford.edu/class/cs224n/readings/cs224n-2019-notes02-wordvecs2.pdf

Word analogy: Comparison

Comparison of the performance of different models under the use of different hyperparameters and datasets

Model	Dimension	Size	Semantics	Syntax	Total
ivLBL	100	1.5B	55.9	50.1	53.2
HPCA	100	1.6B	4.2	16.4	10.8
GloVE	100	1.6B	67.5	54.3	60.3
SG	300	1B	61	61	61
CBOW	300	1.6B	16.1	52.6	36.1
vLBL	300	1.5B	54.2	64.8	60.0
ivLBL	300	1.5B	65.2	63.0	64.0
GloVe	300	1.6B	80.8	61.5	70.3
SVD	300	6B	6.3	8.1	7.3
SVD-S	300	6B	36.7	46.6	42.1
SVD-L	300	6B	56.6	63.0	60.1
CBOW	300	6B	63.6	67.4	65.7
SG	300	6B	73.0	66.0	69.1
GloVe	300	6B	77.4	67.0	71.7
CBOW	1000	6B	57.3	68.9	63.7
SG	1000	6B	66.1	65.1	65.6
SVD-L	300	42B	38.4	58.2	49.2
GloVe	300	42B	81.9	69.3	75.0

Ke observations:

- Performance is heavily dependent on the model used for word embedding
- Performance increases with larger corpus sizes:
- Performance is lower for extremely low dimensional word vector.

The contents of this slides are taken from :
https://web.stanford.edu/class/cs224n/readings/cs224n-2019-notes02-wordvecs2.pdf

Intrinsic Evaluation: Word Similarity

- Measures how well word embeddings reflect **human judgments** of semantic similarity between words.
- Example: "car" and "automobile" should have **high similarity**, "car" and "banana" should have **low similarity**.

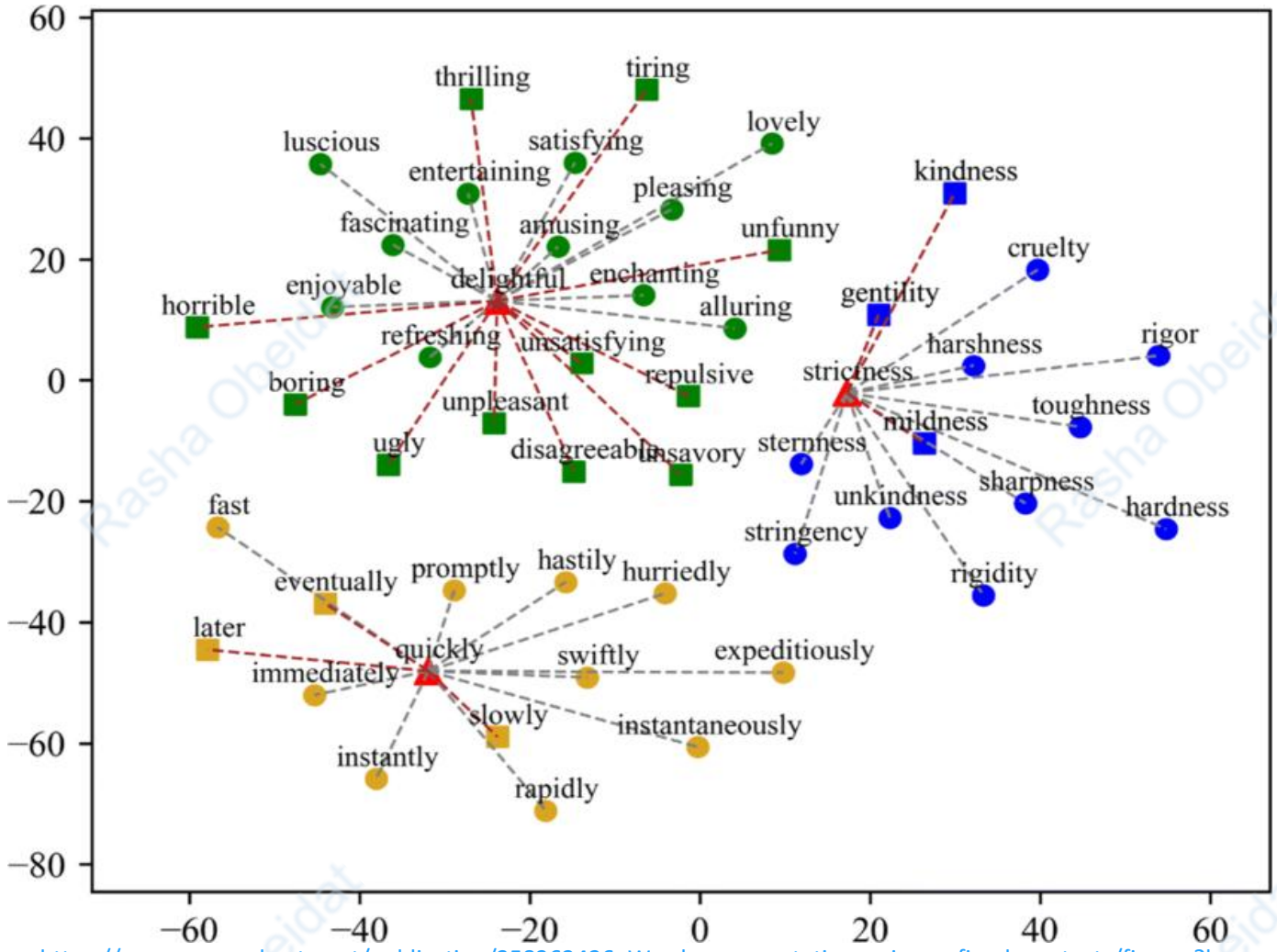
How It Works:

1. A dataset contains **pairs of words** with human-assigned similarity scores.
2. The model computes **cosine similarity** between the embeddings of each word pair.
3. The correlation (typically Spearman or Pearson) between model scores and human scores is reported; correlation means the model's similarity rankings **align well with human intuition**.

Example Datasets: WordSim-353, SimLex-999, MEN

Intrinsic Evaluation: Clustering & Visualization

Use t-SNE or PCA to reduce dimensionality and plot word vectors, then ***visually*** inspect if similar words cluster together.



https://www.researchgate.net/publication/358362426_Word_representation_using_refined_contexts/figures?lo=1&utm_source=google&utm_medium=organic

Extrinsic Evaluation of word embeddings

- **Extrinsic evaluation of word embeddings** measures their effectiveness in real-world NLP tasks by testing how well they improve model performance using some evaluation metrics (e.g., accuracy, F1 score, BLEU scores)
- **Common Extrinsic Evaluation Tasks**
  - **Text Classification**
    - Use embeddings as input for sentiment analysis, topic classification, spam detection.
    - Example: Train a classifier on movie reviews using embeddings.
  - **Named Entity Recognition (NER)**
    - Test embeddings on entity detection tasks.
    - Example: Identifying "New York" as a location in text.
  - **Machine Translation (MT)**
    - Use embeddings in translation models to improve quality.
    - Compare BLEU scores between models using different embeddings.
  - **Question Answering (QA)**
    - Test embeddings on reading comprehension datasets (e.g., SQuAD, ArQuad).
    - Compare BLEU scores between models using different embeddings.

Suggested Readings

- http://web.stanford.edu/class/cs224n/readings/cs224n-2019-notes01-wordvecs1.pdf
- https://d2l.ai/chapter_natural-language-processing-pretraining/word2vec.html

- These slides are the intellectual property of **Dr. Rasha Obeidat®**. If you wish to use them for teaching **your course**, please contact the author for permission.
  The slides may **not be modified or redistributed** without explicit permission.
- **Contact:**
  Obeidat.rm@gmail.com
  Rmobeidat@just.edu.jo



Rasha Obeidat